African Scientific Research and Innovation Council

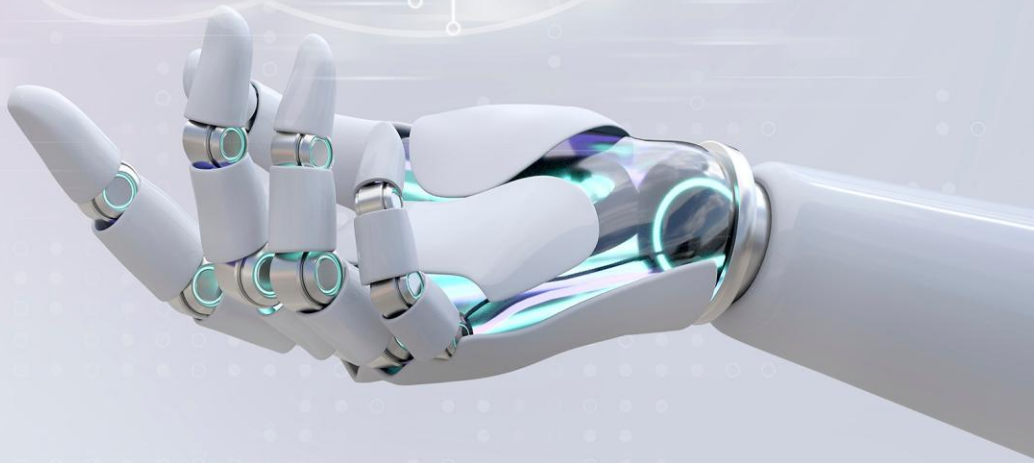
BAAI 智源研究院

北京大学 PEKING UNIVERSITY

中国人民大学 RENMIN UNIVERSITY OF CHINA

Global AI Compute Faculty Development Initiative

Module 02: High-Performance AI Operators and Operator Engineering



Guojie Luo gluo@pku.edu.cn

Jing Wang jwang@ruc.edu.cn

October 2026 · 2nd AICFD

Supported by UNDP

Outline

1. Attention and FlashAttention

2. Operator Fusion and Case Study

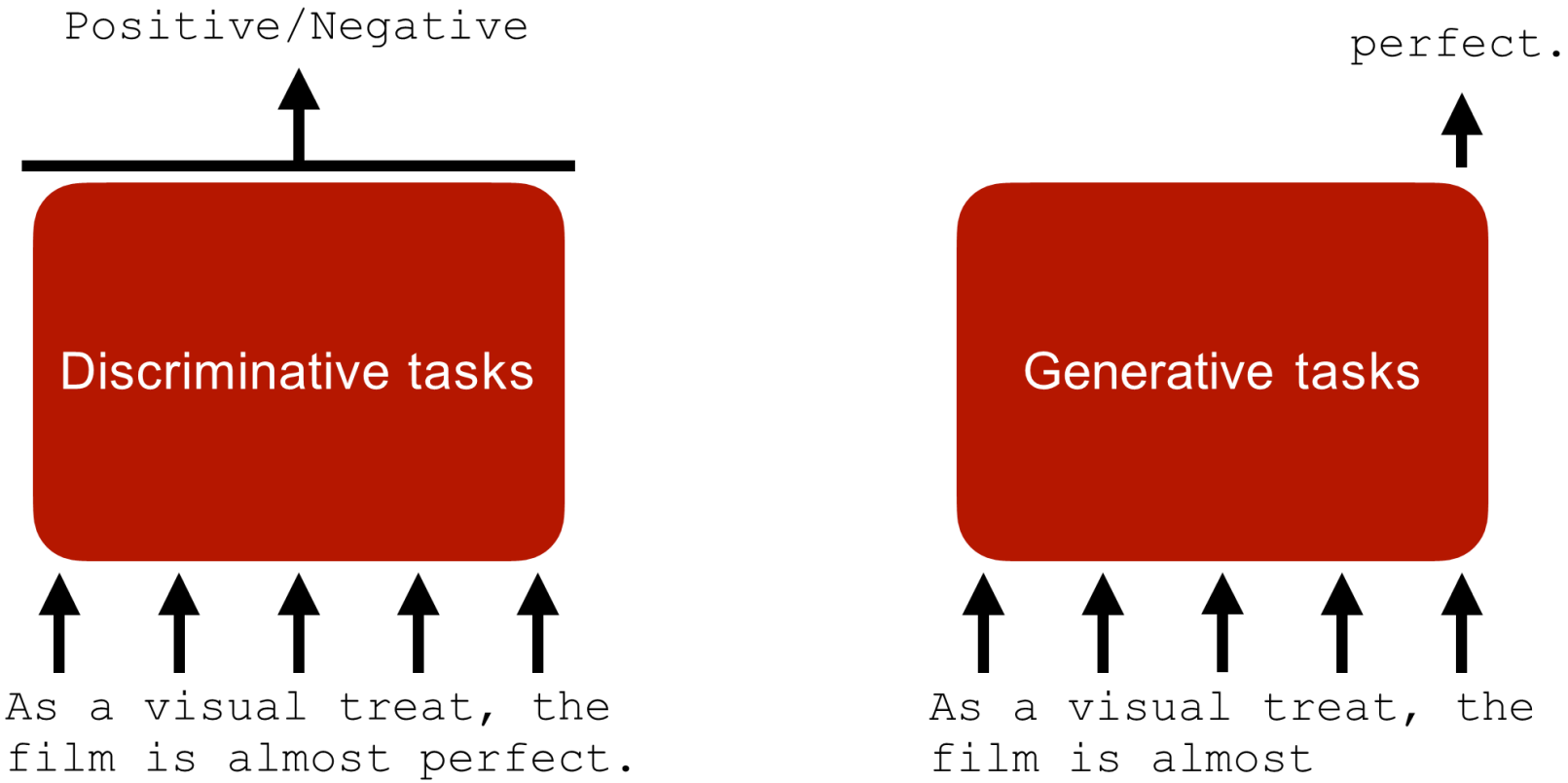
3. Architectural Design of Operator Libraries

1. Attention and FlashAttention

- Transformer (2017) at a Glance
- Self-Attention Builds Context
- Q, K, and V: Match, Score, and Mix
- Why Use Multiple Attention Heads?
- Causal Masking Blocks Future Tokens
- The FFN Transforms Each Token
- Residual Connections Keep a Direct Path
- LayerNorm Stabilizes Token Features

1.1. Revisit: NLP Tasks

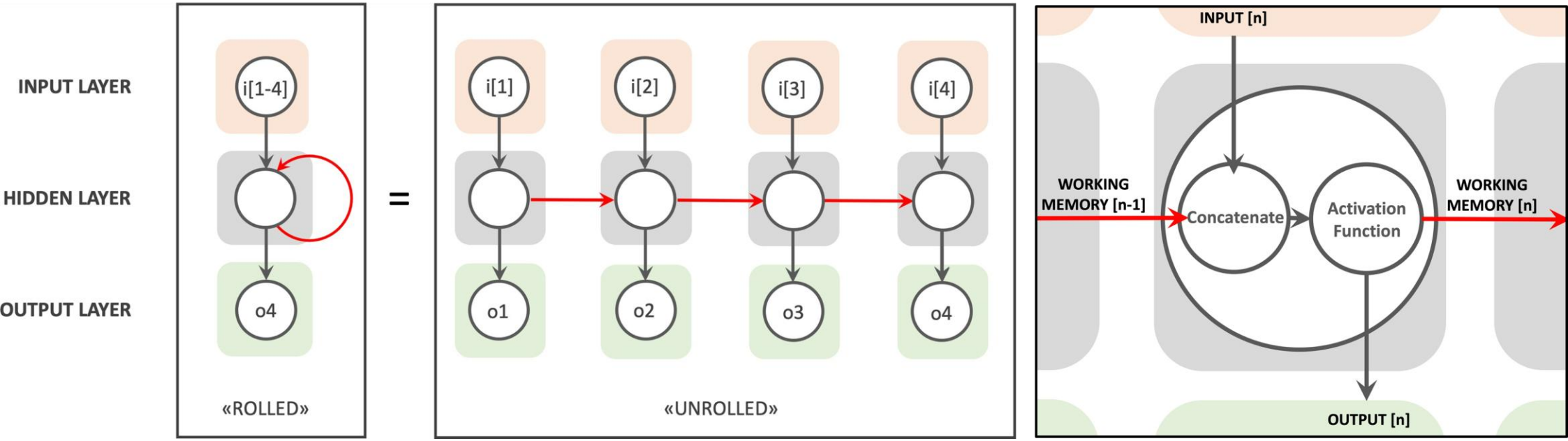
- **Discriminative tasks** (e.g., sentiment analysis, text classification, textual Entailment)
- **Generative tasks** (e.g., language modeling, machine translation, summarization)



1.2. Pre-Transformer Era

Recurrent Neural Networks (RNNs)

- The "working memory" struggles to retain **long-term** dependancies (can be solved by LSTM).
- There is **dependency** across tokens, limiting the scalability.



1.3. Pre-Transformer Era

Convolutional Neural Networks (CNNs)

- No dependency between tokens, leading to **better scalability**.
- **Limited context information**, resulting in worse modeling capability.

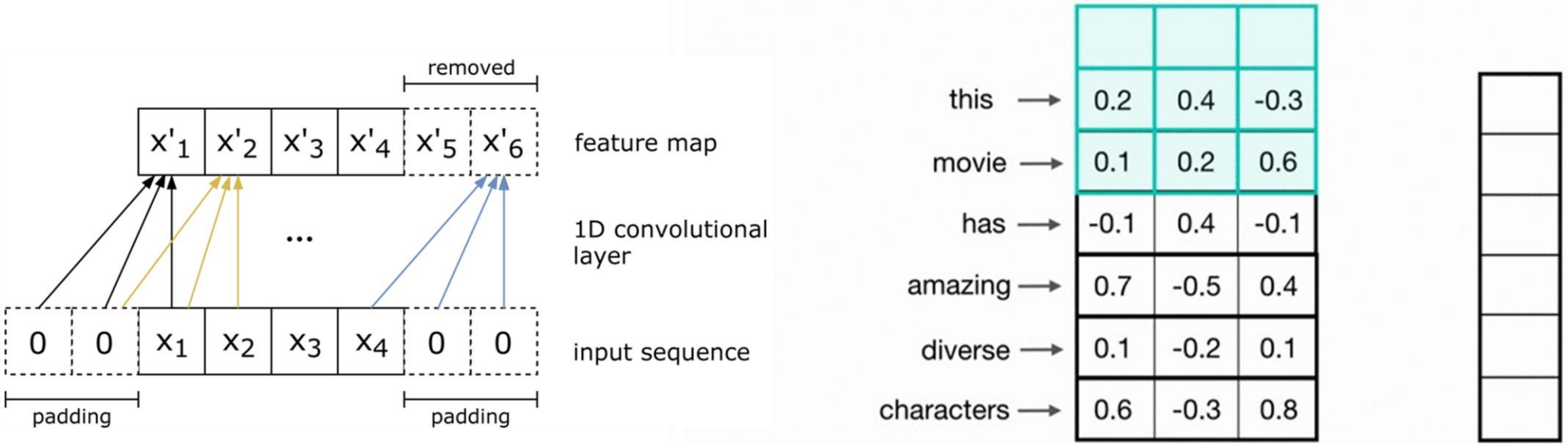
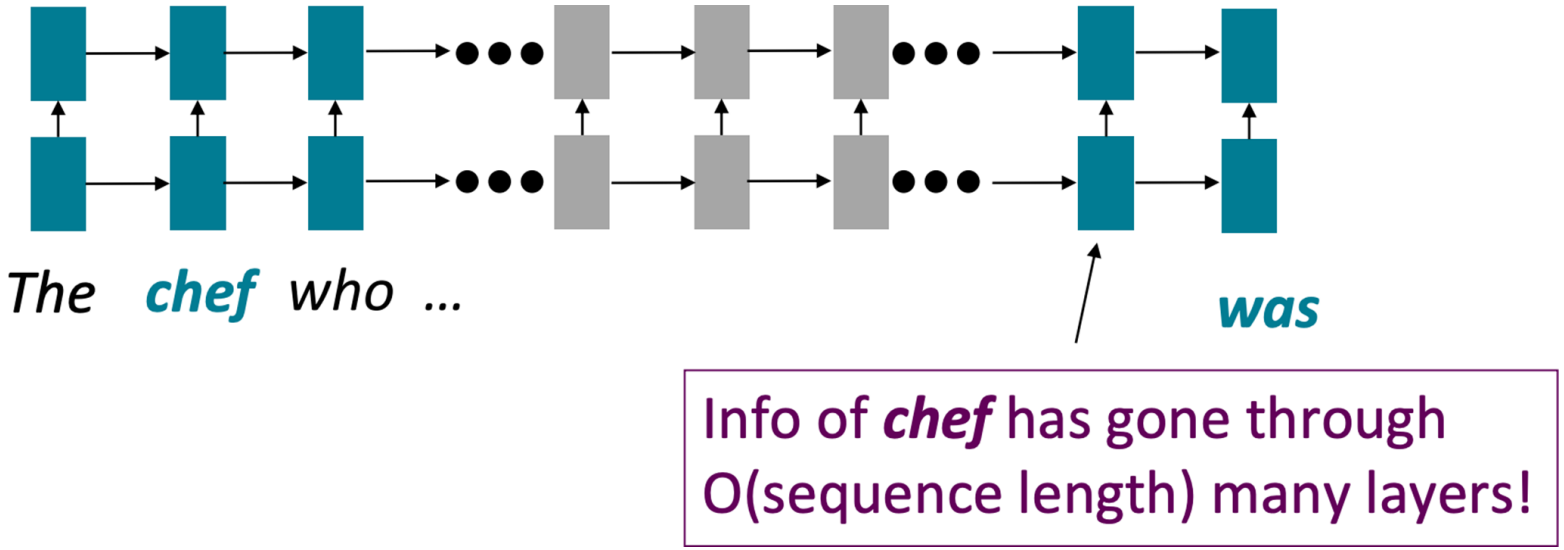


Image credit: https://cezannec.github.io/CNN_Text_Classification/

1.4. Pre-Transformer Era

Problem with RNNs/LSTMs

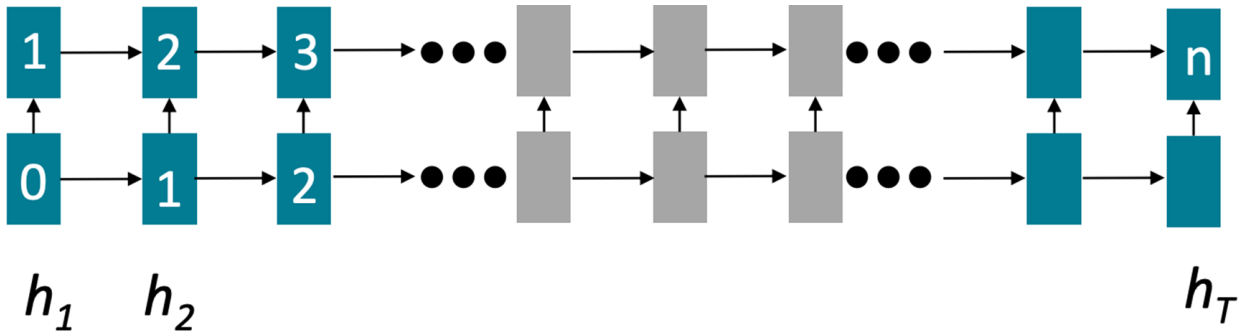
- **Hard to model long-term relationships**
  - It takes O(seq_len) steps to model the interaction between two tokens
  - Images have locality; but languages may not have locality:



1.5. Pre-Transformer Era

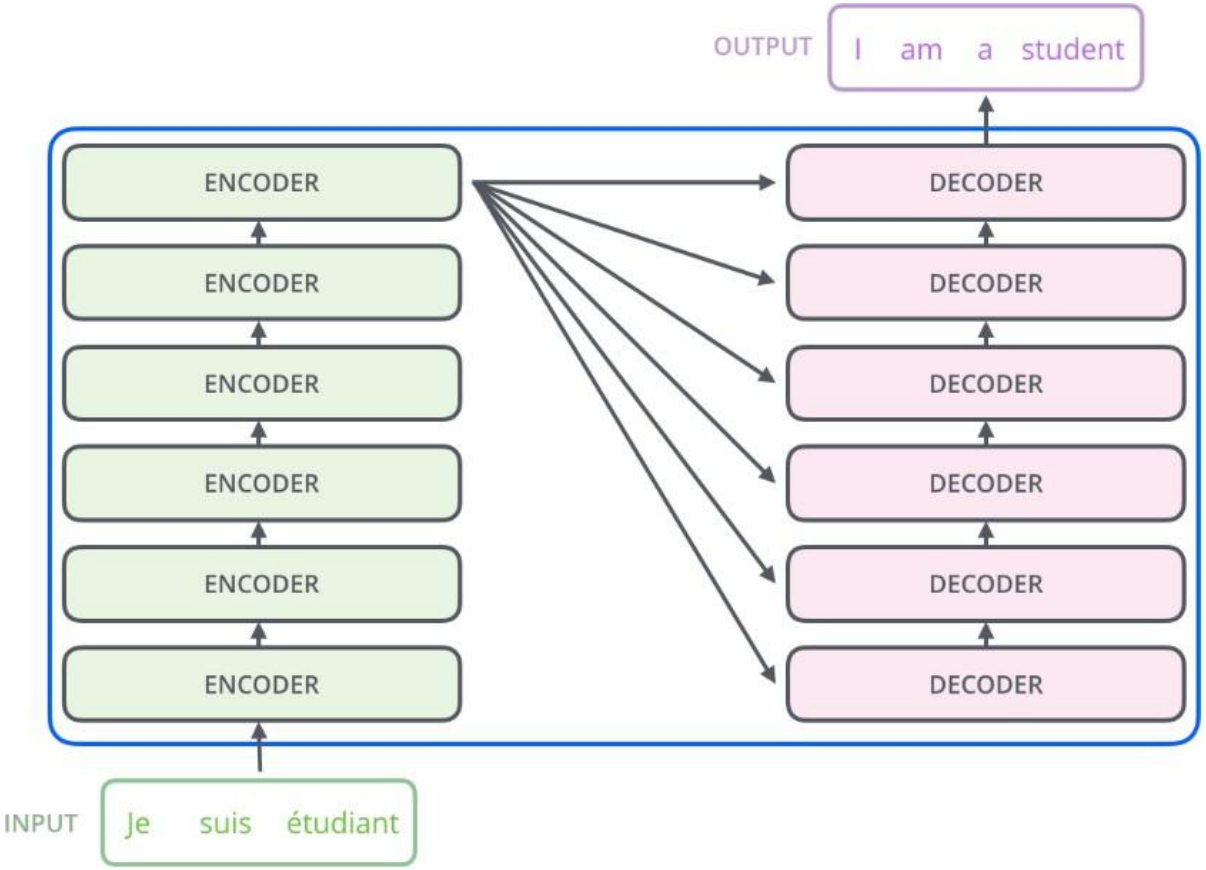
Problem with RNNs/LSTMs

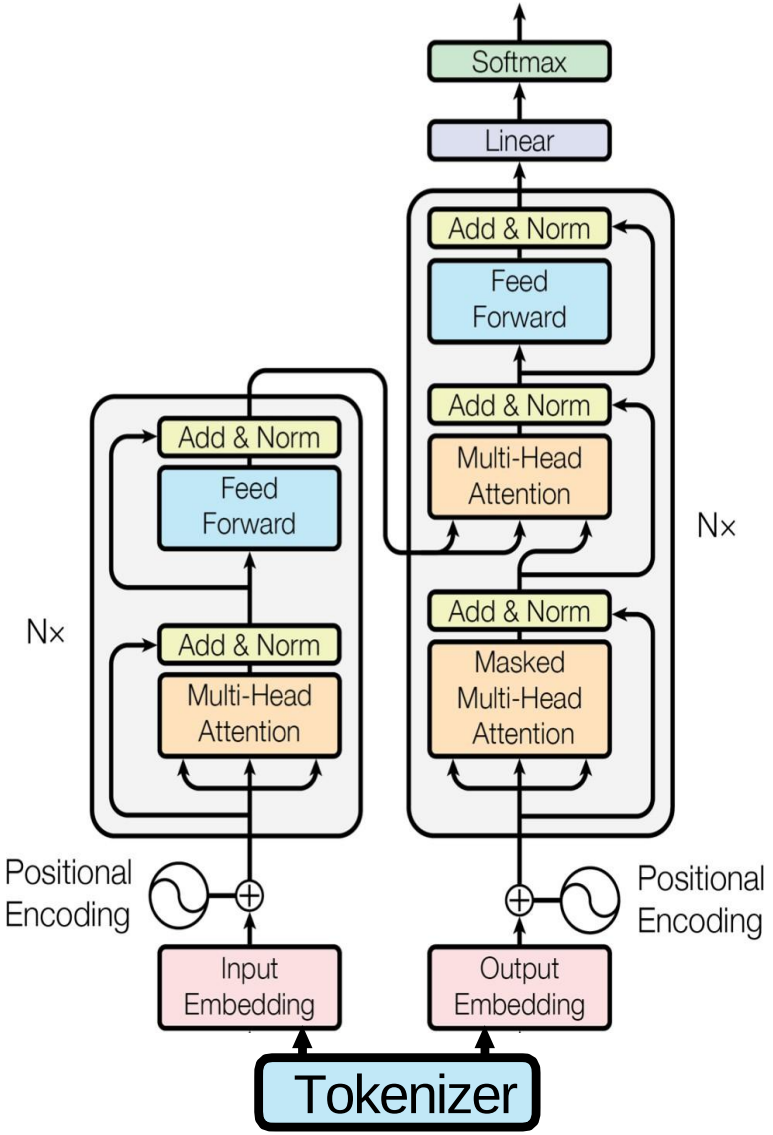
- **Hard to model long-term relationships**
- **Limited training parallelism**
  - The states are strictly dependent on earlier states
  - It takes *n* steps to get state *n*



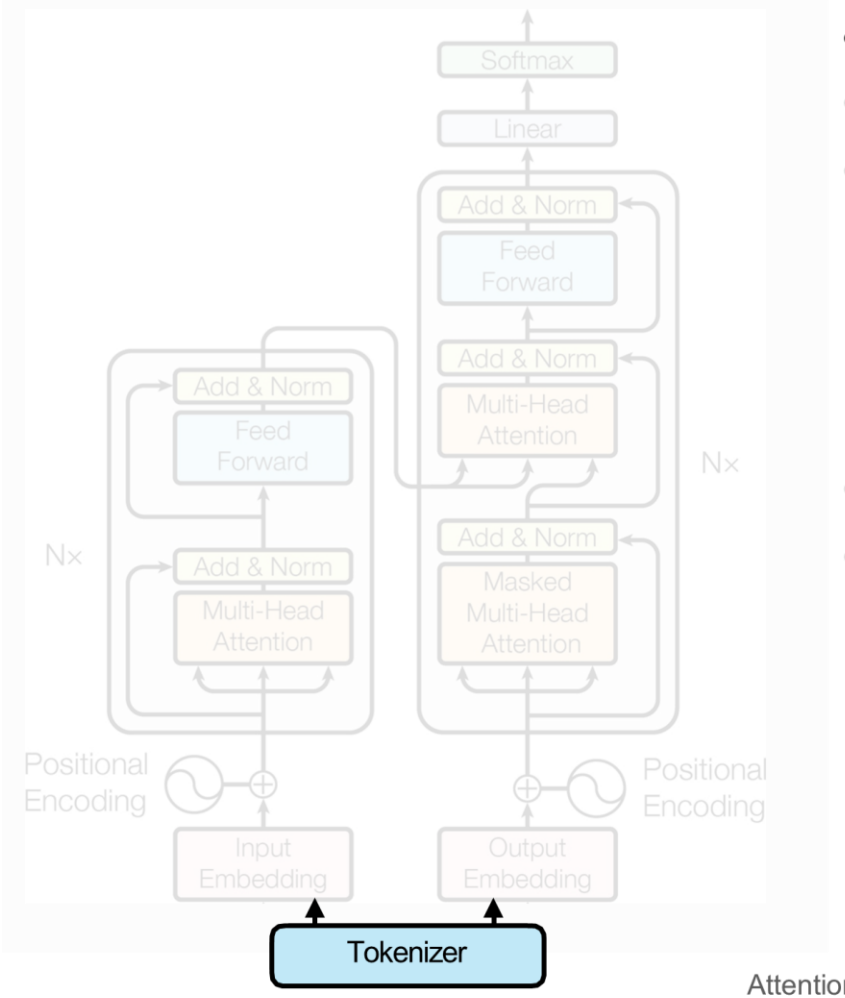
Numbers indicate min # of steps before a state can be computed

1.6. Transformer (2017) at a Glance





1.7. Transformer (2017) at a Glance



- Tokenize words (word -> tokens)
- Map tokens into embeddings
- Embeddings go through transformer blocks
  - Multi-Head Attention (MHA)
  - Feed-Forward Network (FFN)
  - LayerNorm
  - Residual connection
- Positional encoding
- Final prediction with linear head

Attention Is All You Need [Vaswani *et al.*, 2017]

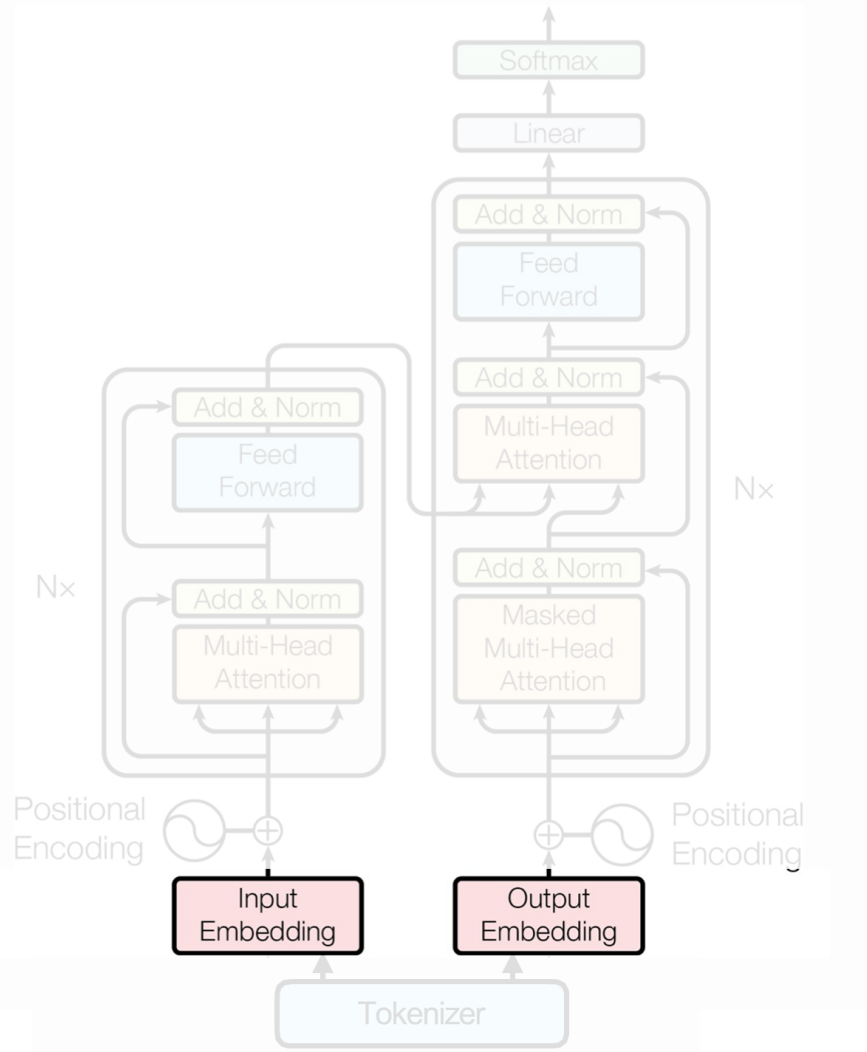
1.8. Tokenization

- A tokenizer maps a word to one/multiple tokens:
- **110 words** => **162 tokens**

Large generative models (e.g., large language models, diffusion models) have shown remarkable performance, but they require a massive amount of computational resources. To make them more accessible, it is crucial to improve their efficiency.This course will introduce efficient AI computing techniques that enable powerful deep learning applications on resource-constrained devices. Topics include model compression, pruning, quantization, neural architecture search, distributed training, data/model parallelism, gradient compression, and on-device fine-tuning. It also introduces application-specific acceleration techniques for large language models, diffusion models, video recognition, and point cloud. This course will also cover topics about quantum machine learning. Students will get hands-on experience deploying large language models (e.g., LLaMA 2) on a laptop.

TEXT TOKEN IDS

1.9. Transformer (2017) at a Glance

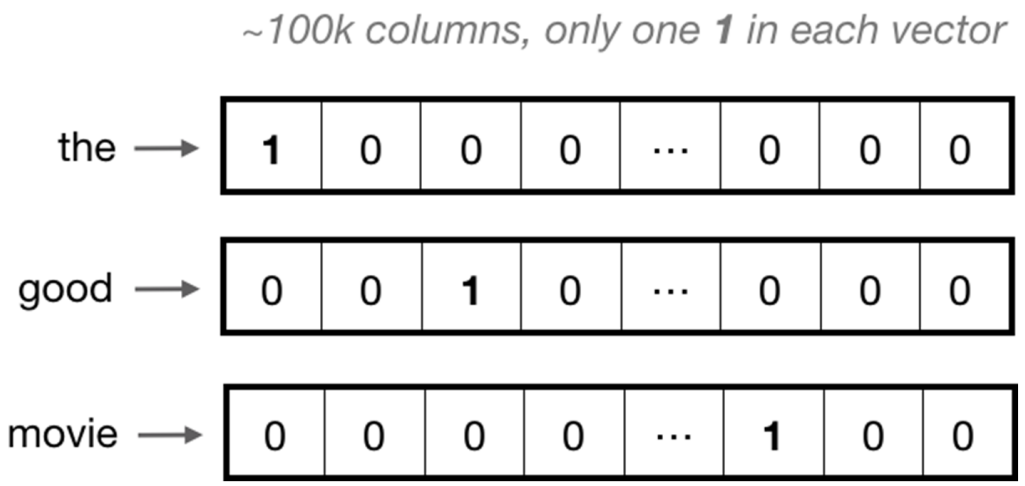


- Tokenize words (word -> tokens)
- Map tokens into embeddings
- Embeddings go through transformer blocks
  - Multi-Head Attention (MHA)
  - Feed-Forward Network (FFN)
  - LayerNorm
  - Residual connection
- Positional encoding
- Final prediction with linear head

1.10. Word Representation

One-Hot Encoding

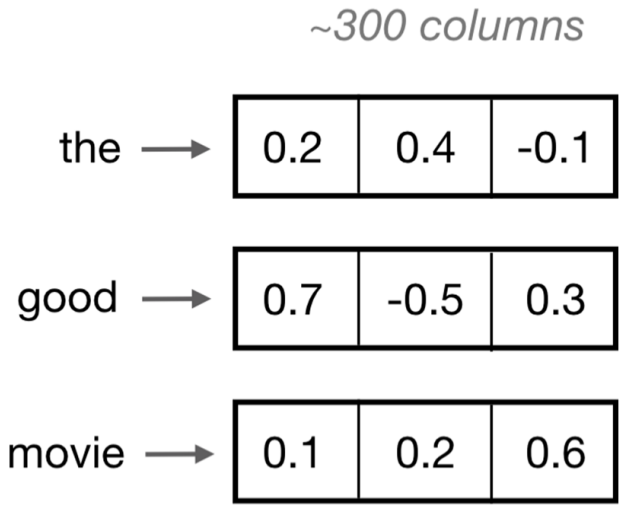
- **Key idea**: Representing each word as a vector that has as many values in it as there are words in the vocabulary. Each column in a vector represents one possible word in a vocabulary.



- For **large vocabularies**, these vectors can get very **long**, and they contain all 0's except for one value. This is considered a very sparse representation.

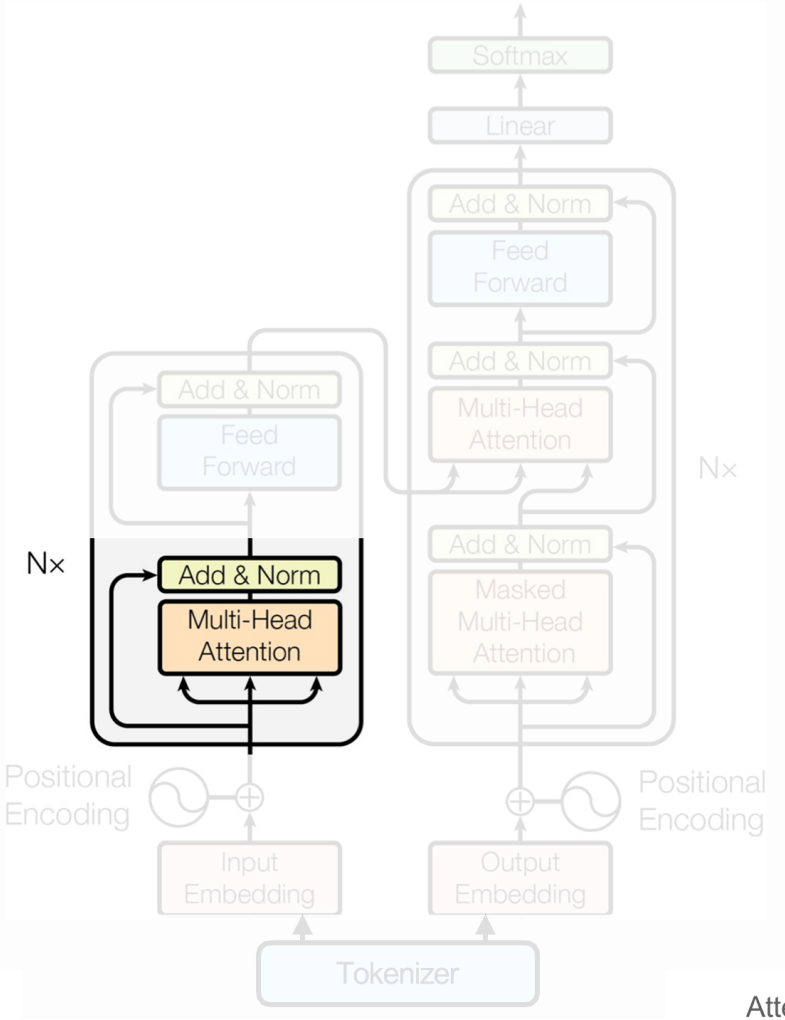
1.11. Word Representation

- **Key idea**: Map the word index to a **continuous** word embedding through a **look-up table**.



- The word embedding can be trained **end-to-end** with the model for the downstream tasks.
  - Popular pre-trained word embeddings: Word2Vec, GloVe.

1.12. Transformer (2017) at a Glance

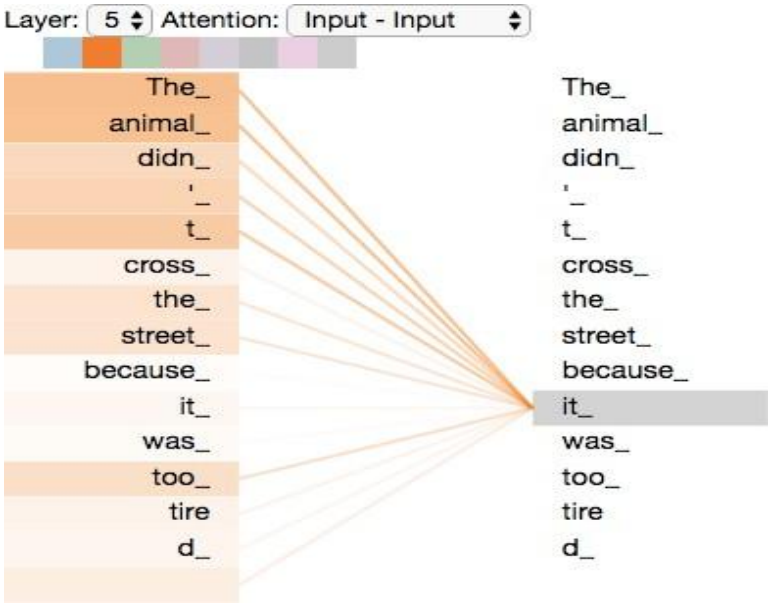


- Tokenize words (word -> tokens)
- Map tokens into embeddings
- Embeddings go through transformer blocks
  - Multi-Head Attention (MHA)
  - Feed-Forward Network (FFN)
  - LayerNorm
  - Residual connection
- Positional encoding
- Final prediction with linear head

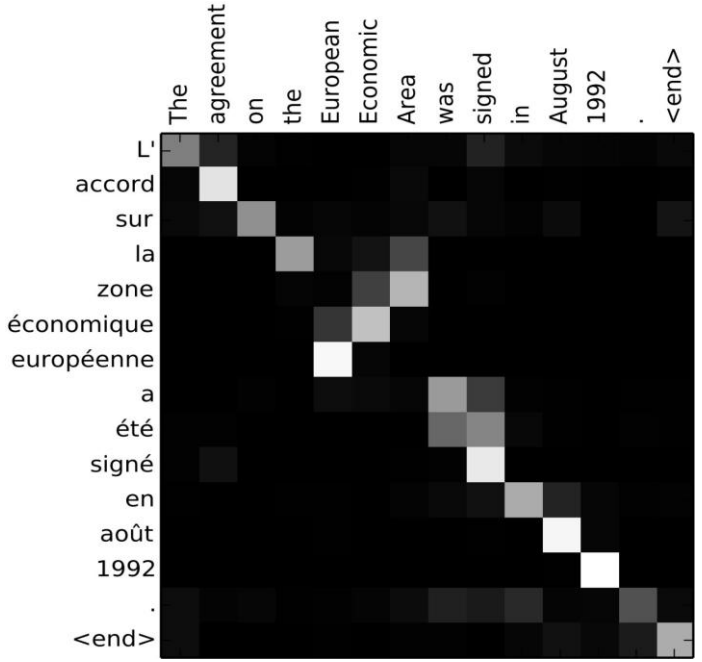
Attention Is All You Need [Vaswani *et al.*, 2017]

1.13. Self-Attention Builds Context

- Each token gathers information from relevant tokens in the same sequence.
- The output is contextual–not an isolated word embedding.



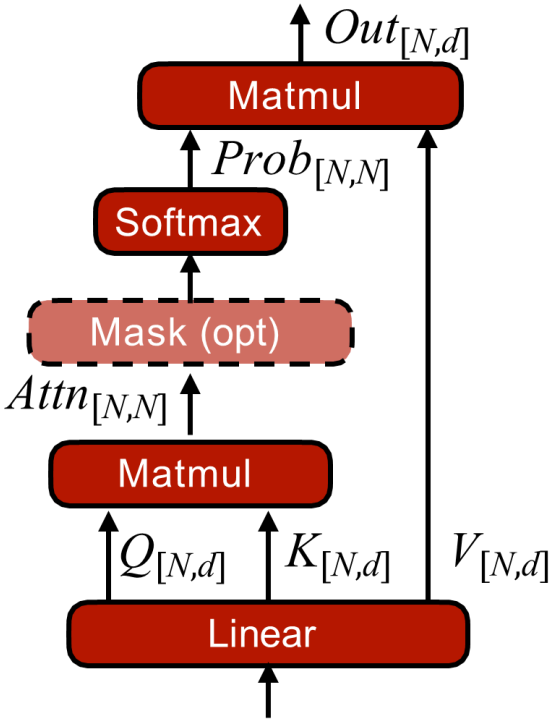
- Example of attention matrix from machine translation
- The attention mechanism finds the correspondence between words



1.14. Q, K, and V: Match, Score, and Mix

- Project the embedding into query, key, and value (Q, K, V)
- The Query-Key-Value design is analogous to a retrieval system (let's take YouTube search as an example)
  - Query: text prompt in the search bar
  - Key: the titles/descriptions of videos
  - Value: the corresponding videos

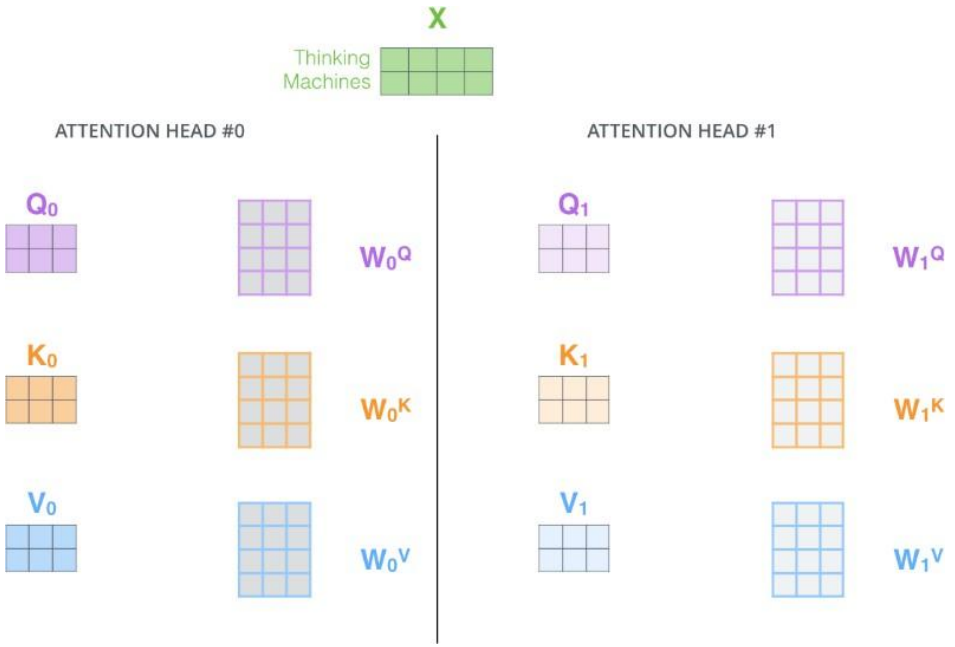
$$\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$$

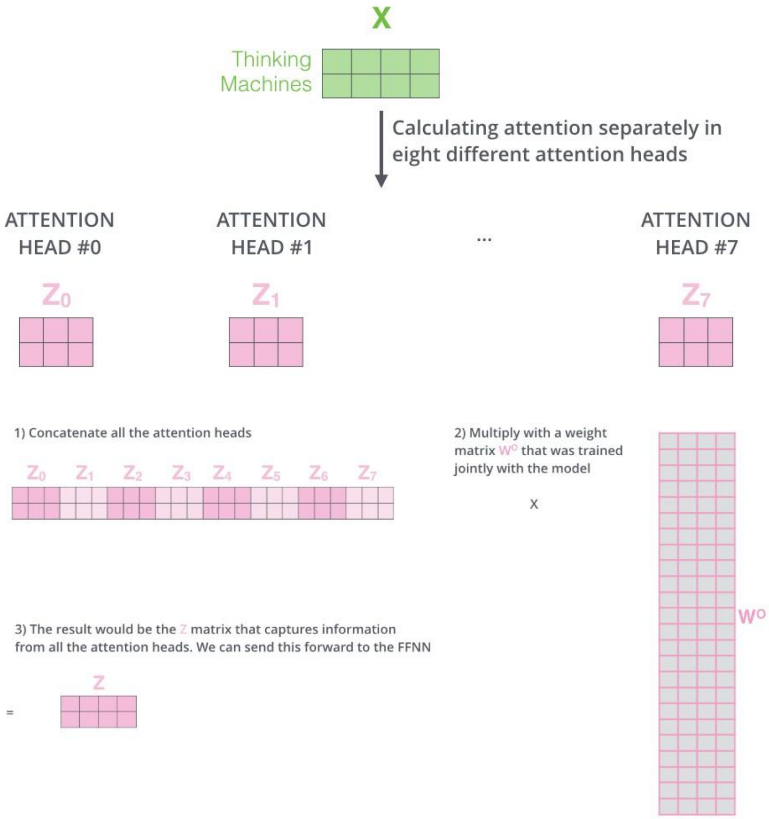


- Multiply Q and to get the inner product (normalize by $\sqrt{d}$)
- Pass through Softmax to get the attention weights of shape $N \times N$
  - The attention computation has $O(N^2)$ complexity!
-

1.15. Why Use Multiple Attention Heads?

- Each head runs attention in a smaller representation space.
- Head outputs are concatenated and projected back to the model dimension.

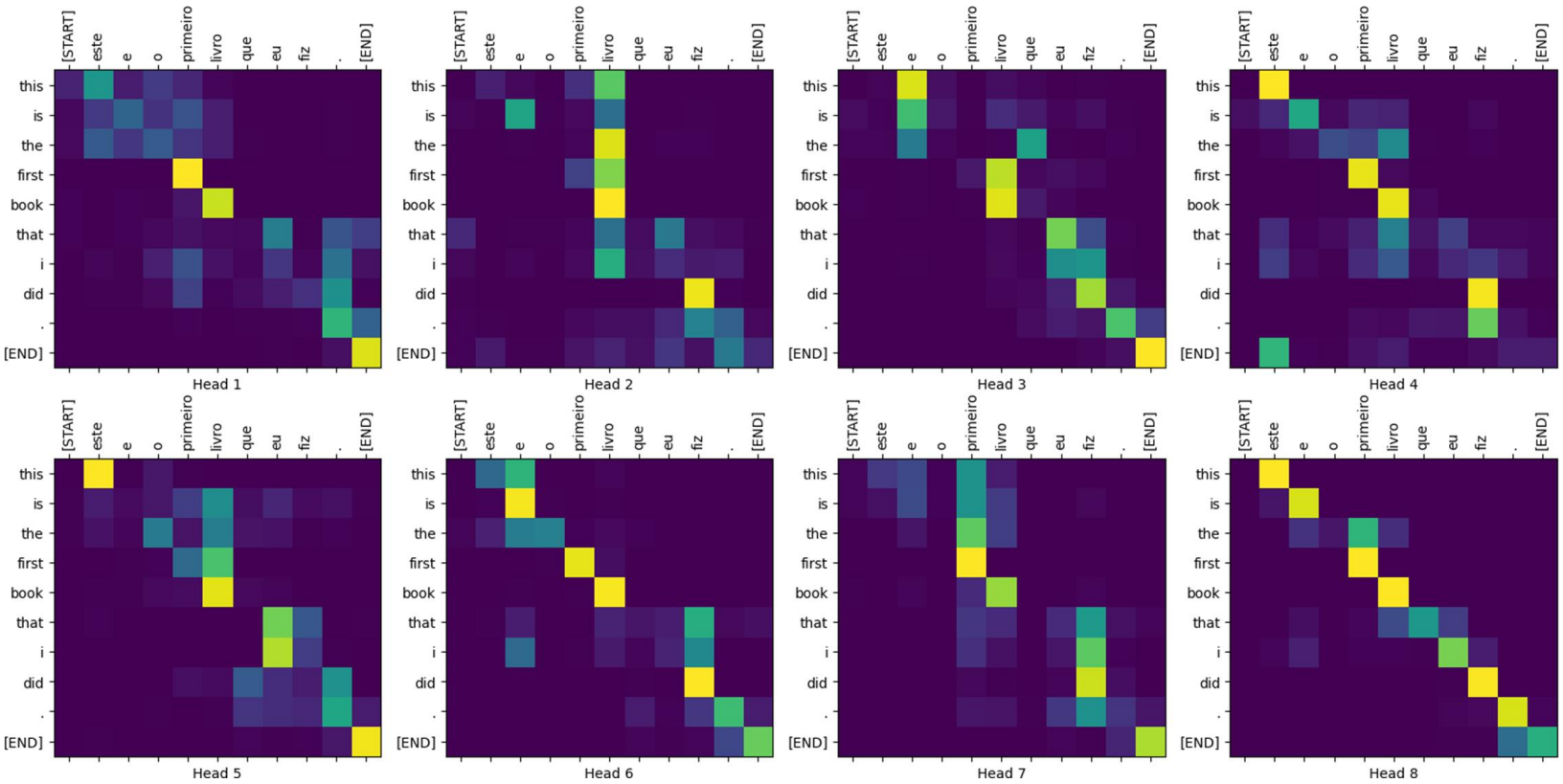




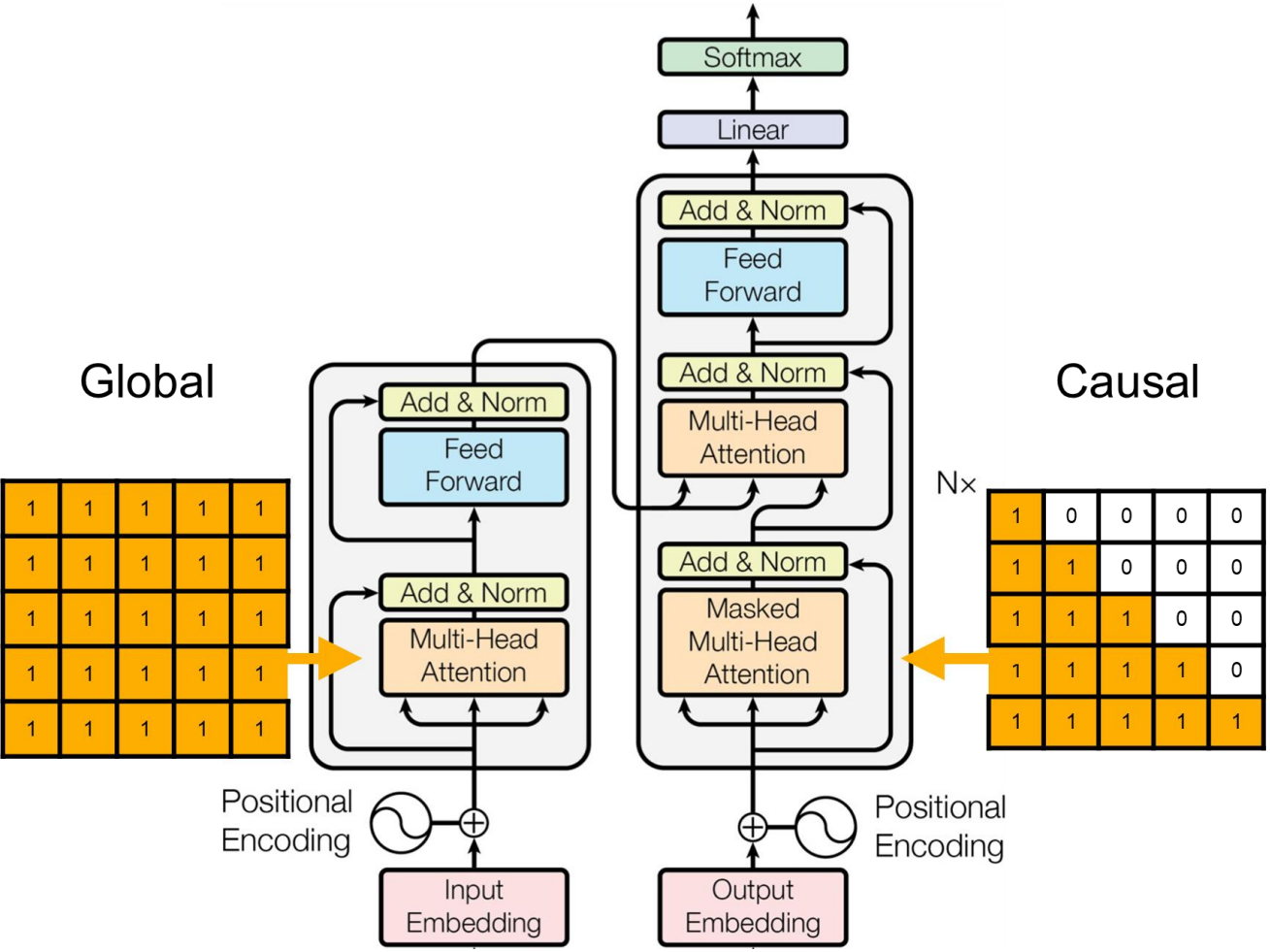
1.16. Why Use Multiple Attention Heads?

Each head captures different semantics

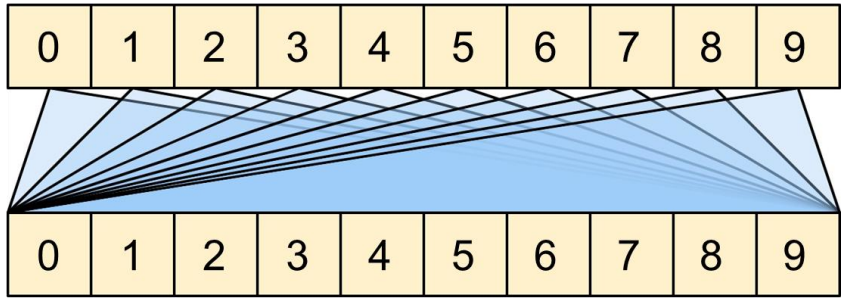
- Visualization from different heads



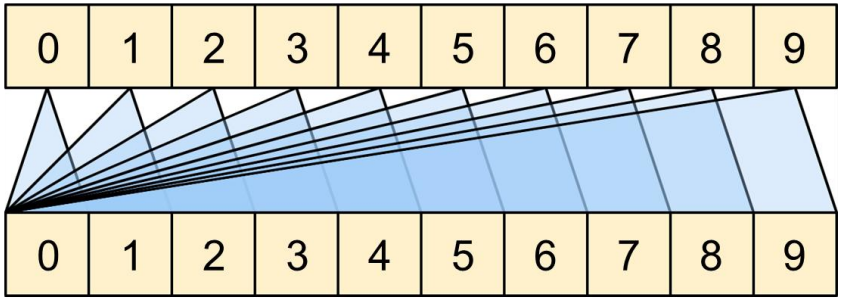
1.17. Causal Masking Blocks Future Tokens



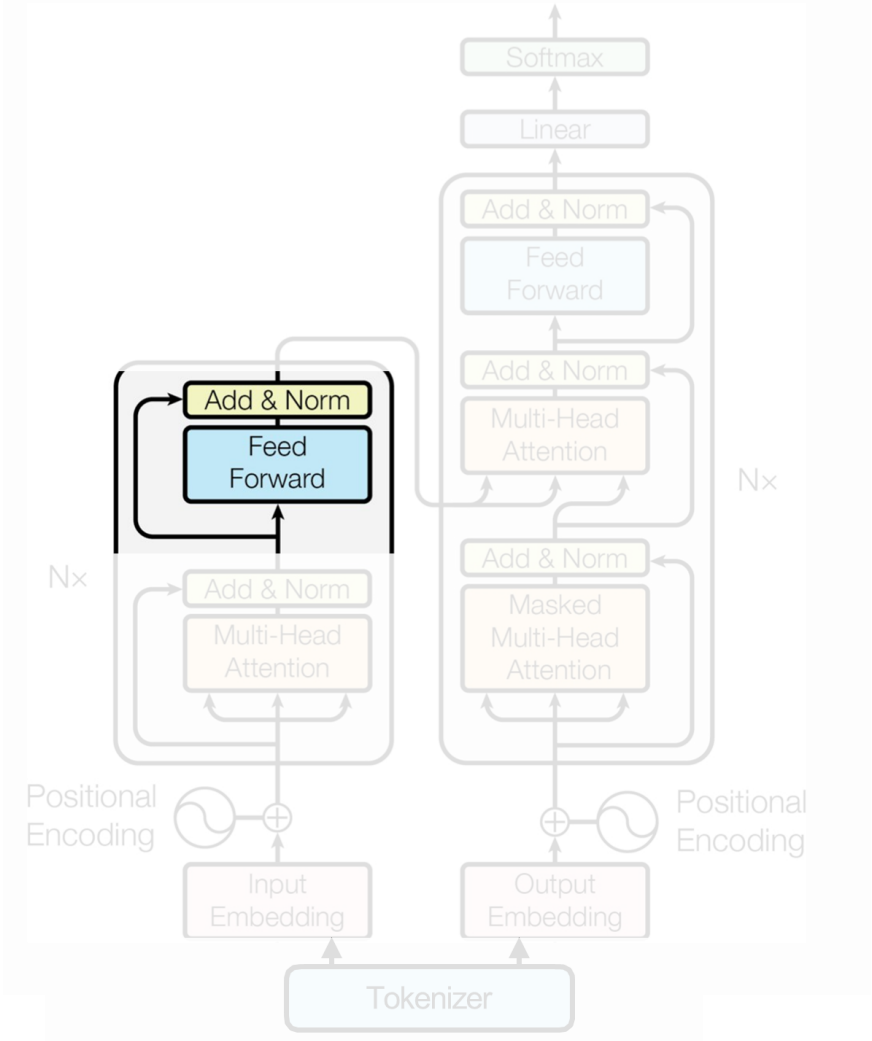
- Global self attention layer



- Causal self attention layer



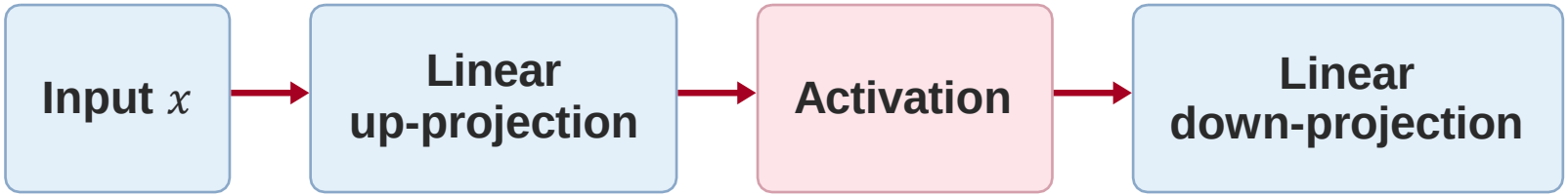
1.18. Transformer



- Tokenize words (word -> tokens)
- Map tokens into embeddings
- Embeddings go through transformer blocks
  - Multi-Head Attention (MHA)
  - Feed-Forward Network (FFN)
  - LayerNorm
  - Residual connection
- Positional encoding
- Final prediction with linear head

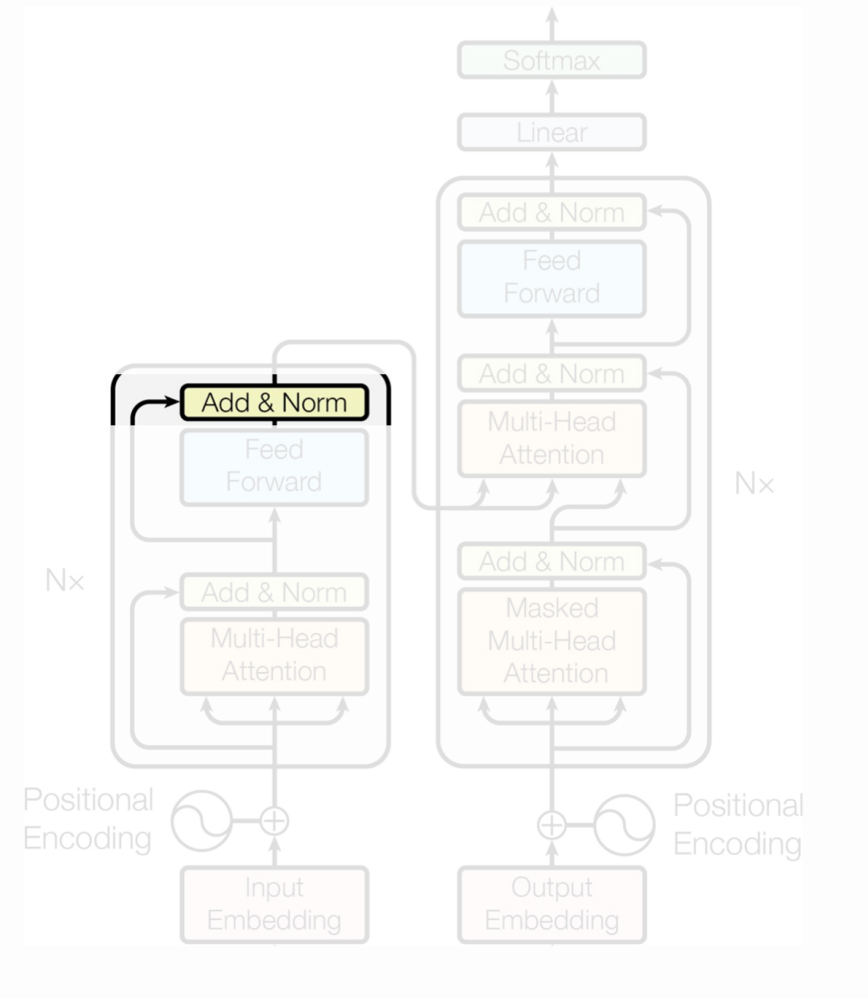
1.19. The FFN Transforms Each Token

- Attention mixes information across tokens;
- the feed-forward network then transforms each token position with the same MLP.
- Add a feed-forward network (FFN) to help with feature modeling



$$\mathrm{FFN}(x) = \mathrm{activation}(xW_1 + b_1)W_2 + b_2$$

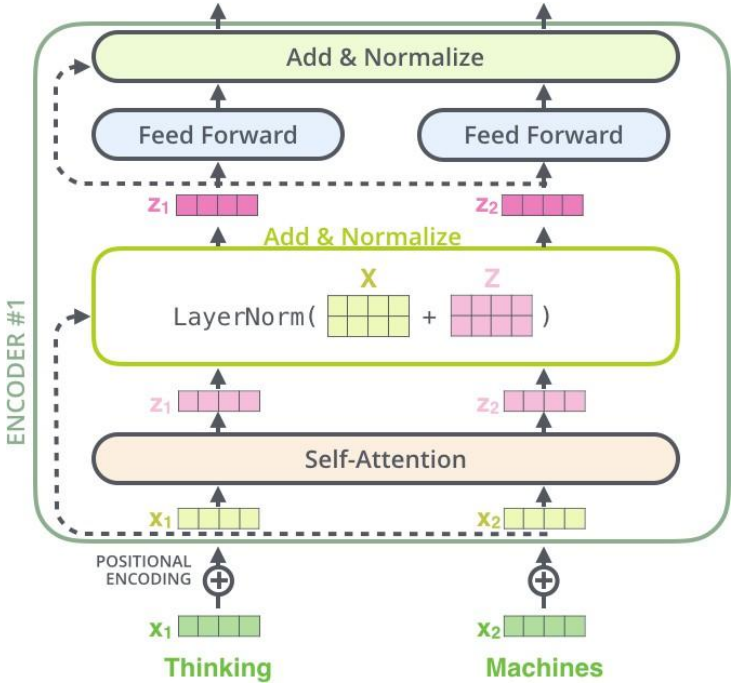
1.20. Transformer

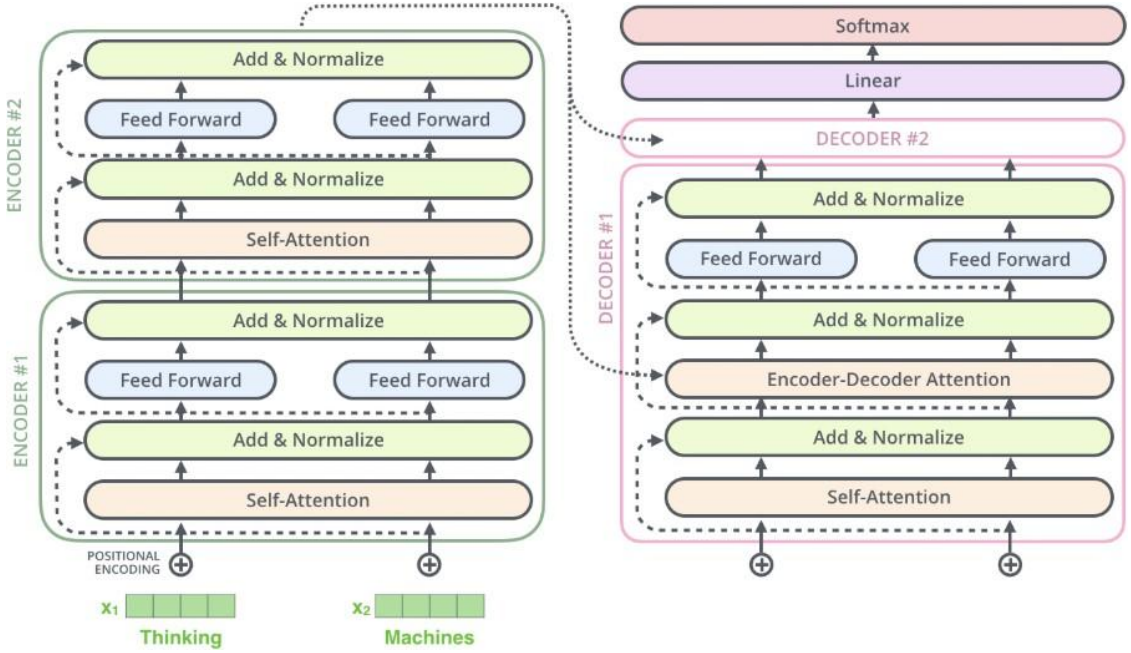


- Tokenize words (word -> tokens)
- Map tokens into embeddings
- Embeddings go through transformer blocks
  - Multi-Head Attention (MHA)
  - Feed-Forward Network (FFN)
  - LayerNorm
  - Residual connection
- Positional encoding
- Final prediction with linear head

1.21. Residual Connections Keep a Direct Path

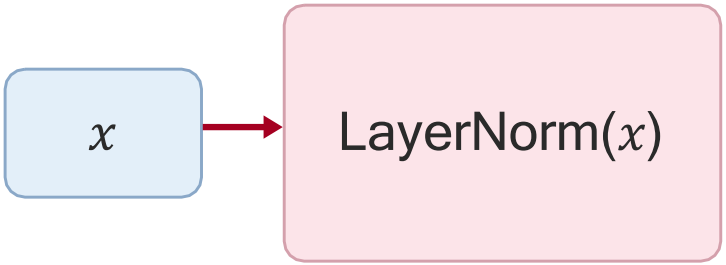
- The shortcut improves information and gradient flow through deep stacks.
- Every sublayer is wrapped by a skip connection.



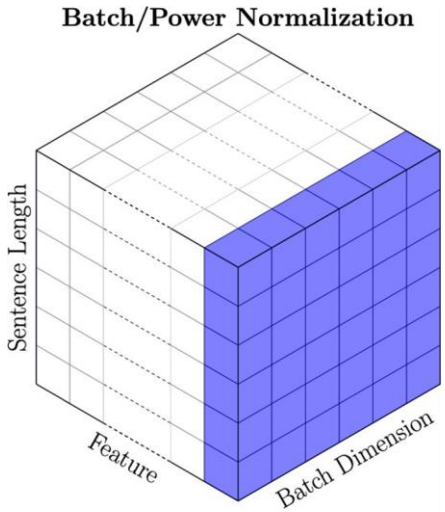


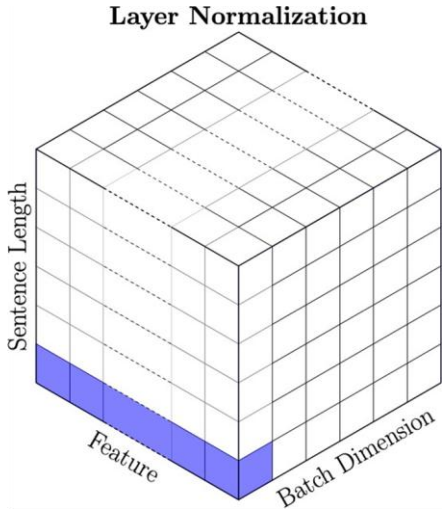
1.22. LayerNorm Stabilizes Token Features

- Normalize across the hidden features of each token.
- Then apply learned scale and bias.



Stabilizes the values passed to the next sublayer.



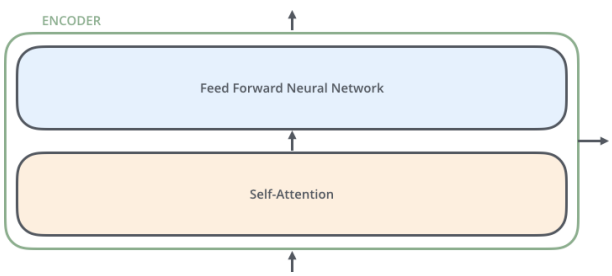


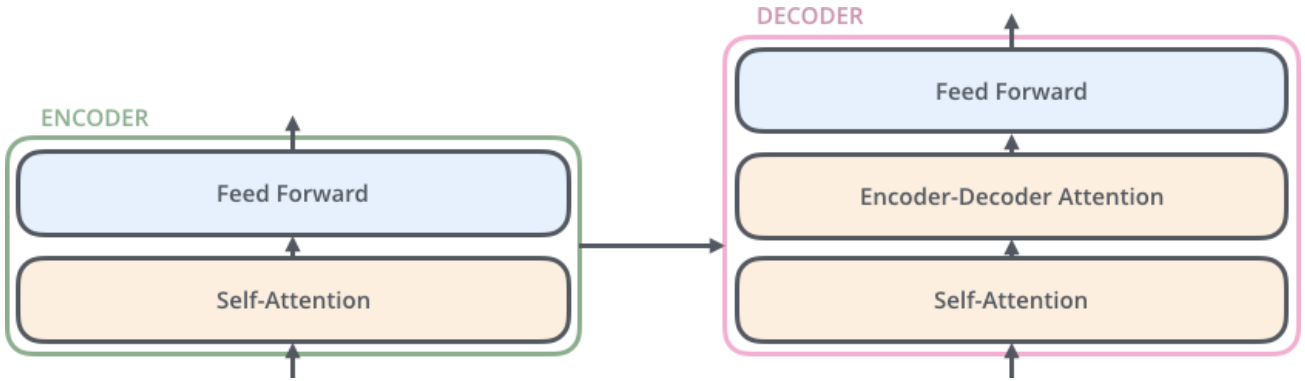
Normalize → Scale (γ) → Shift (β)

1.23. Inside an Encoder Block

- Position-wise FFN
- Add & LayerNorm

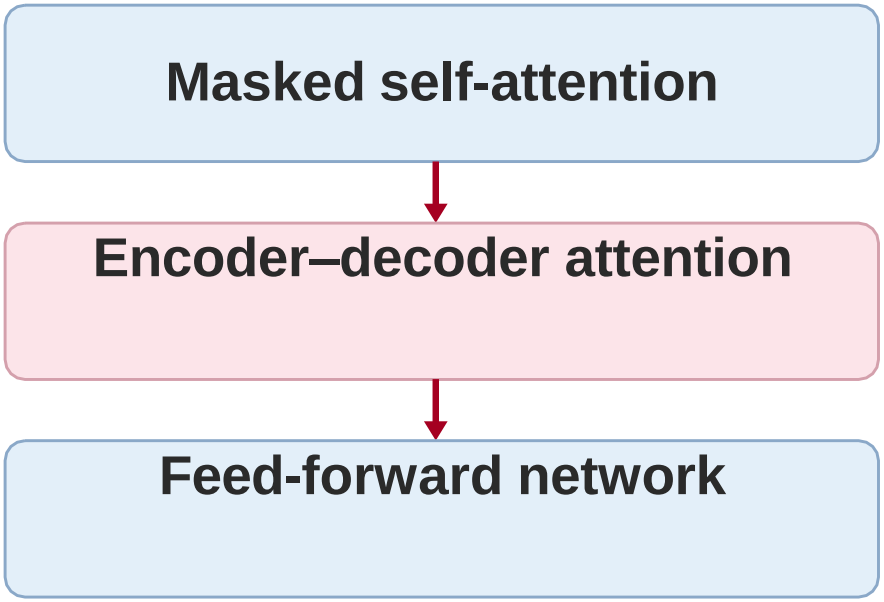
- Multi-head self-attention
- Add & LayerNorm





1.24. Inside a Decoder Block

- **Masked self-attention**
  - Uses only previous output tokens.
- **Cross-attention**
  - Reads the encoder output.
- **Feed-forward network**
  - Transforms each token.

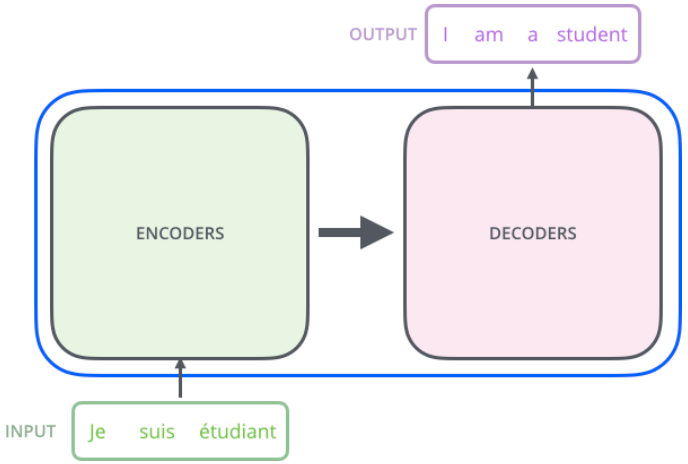


Each sublayer also has residual + normalization

1.25. The 2017 Transformer: Encoder + Decoder

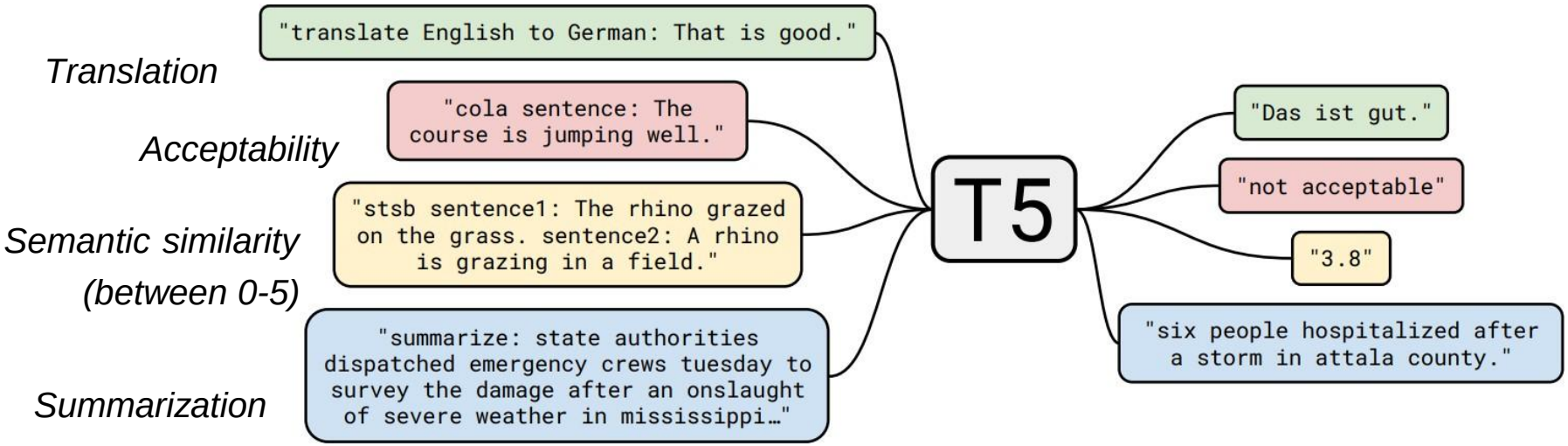
- Encoder reads; decoder generates.





1.26. Encoder-Decoder

- The original Transformer is an Encoder-Decoder architecture
- T5 offers a unified text-to-text model for transfer learning on various NLP tasks



Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer [Raffel *et al.*, 2019]

1.27. Encoder-Decoder

- The original Transformer is an Encoder-Decoder architecture
- T5 offers a unified text-to-text model for transfer learning on various NLP tasks
- The **prompt** is fed to the encoder, the decoder generates the **answer**.

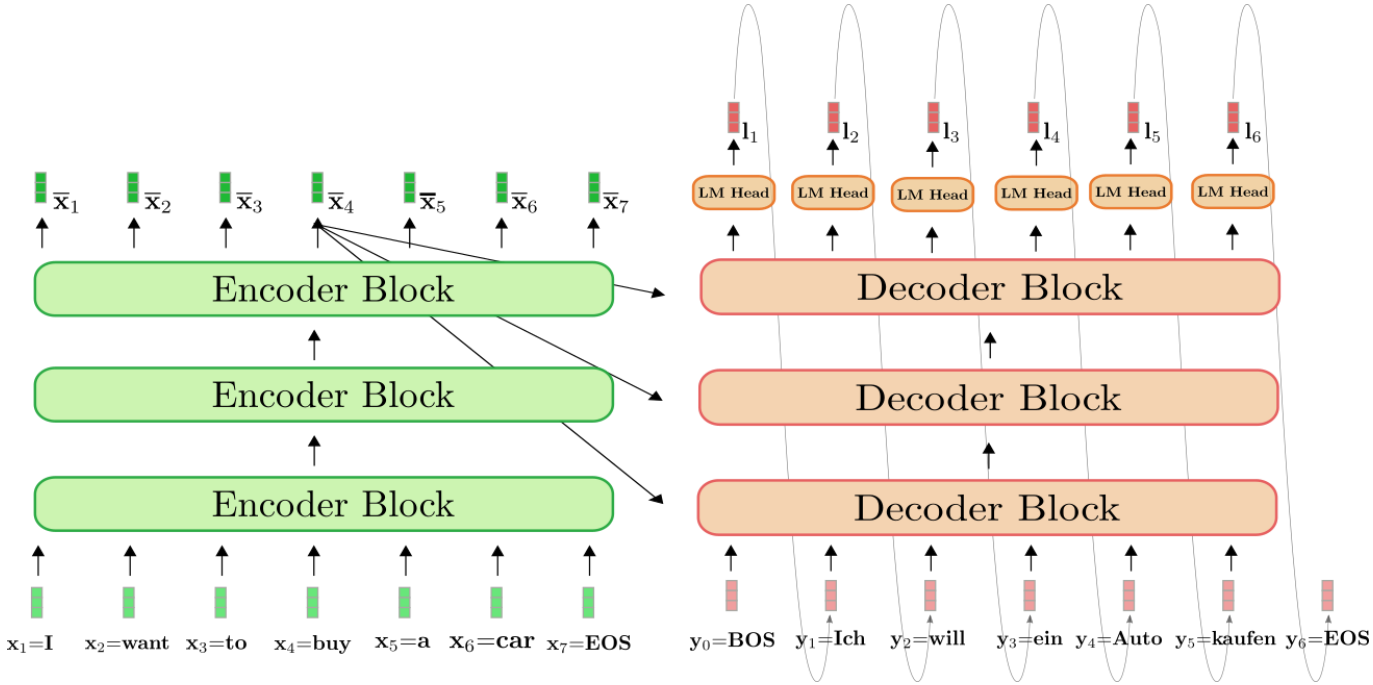


Image credit: https://huggingface.co/blog/encoder-decoder

1.28. Encoder-only (BERT)

Attention mask:

1	1	1	1	1
1	1	1	1	1
1	1	1	1	1
1	1	1	1	1
1	1	1	1	1

Bidirectional Encoder Representations from Transformers

- BERT is an **encoder-only** language model
- Two pre-training objectives
  - Task #1: **Masked Language Model (MLM)**
    - Mask some percentage (15%) of the input tokens at random; train model to predict masked tokens
  - Task #2: **Next Sentence Prediction (NSP)**
    - Whether sentence B is the next sentence of sentence A (less used)
- The pre-trained model is fine-tuned to downstream tasks

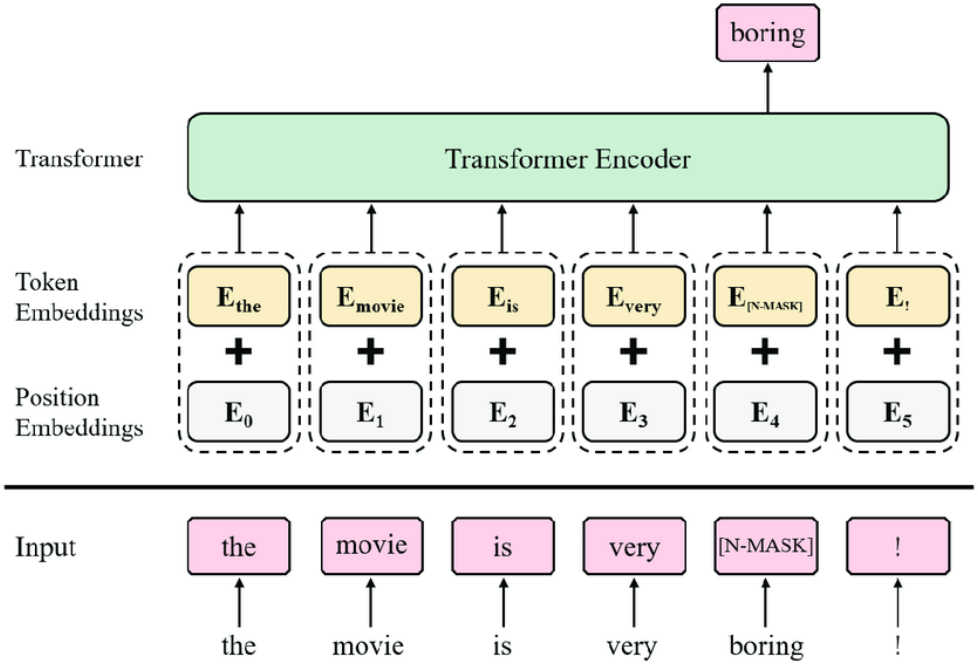


Image credit: https://www.mdpi.com/2073-8994/11/11/1393/htm

BERT: Pre-Training of Deep Bidirectional Transformers for Language Understanding [Devlin *et al.*, 2018]

1.29. Decoder-only (GPT)

Attention mask:

1	0	0	0	0
1	1	0	0	0
1	1	1	0	0
1	1	1	1	0
1	1	1	1	1

Generative Pre-trained Transformer (GPT)

- GPT is a **decoder-only** language model
- Pre-training objective:
  - Next-token prediction

$$L_1(\mathcal{U}) = \sum_i \log P(u_i | u_{i-k}, \ldots, u_{i-1}; \Theta)$$

- For smaller models (GPT-2), the pre-trained model is fine-tuned to downstream tasks
- Larger models support zero-shot and few-shot use

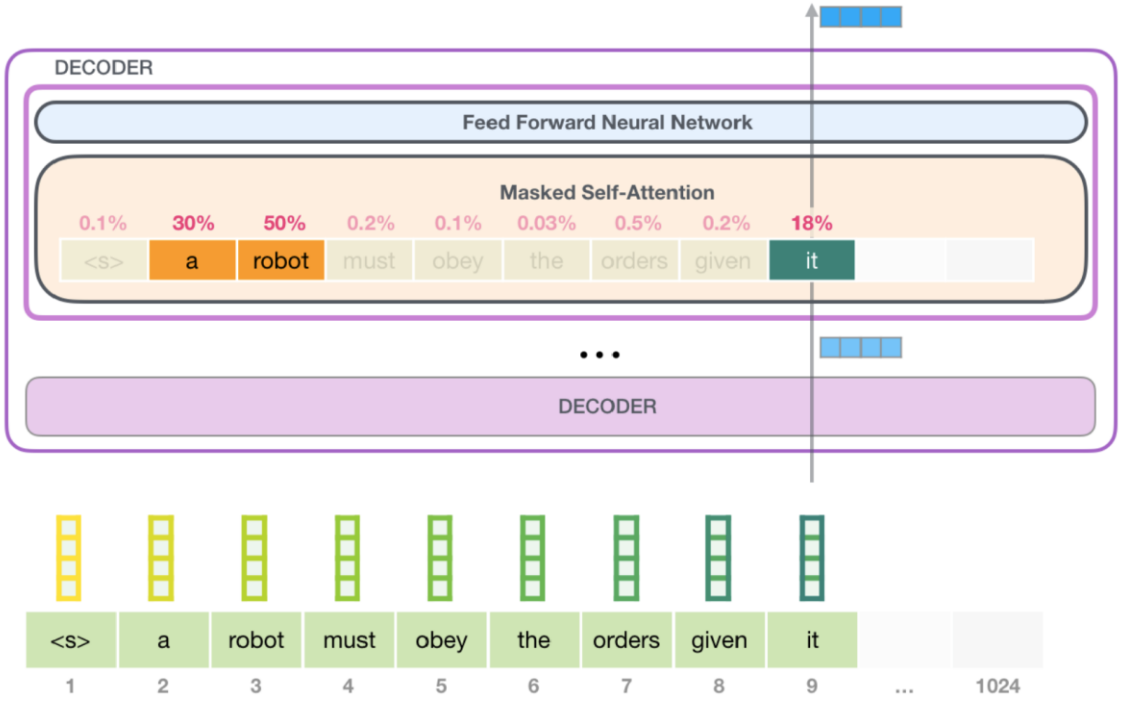
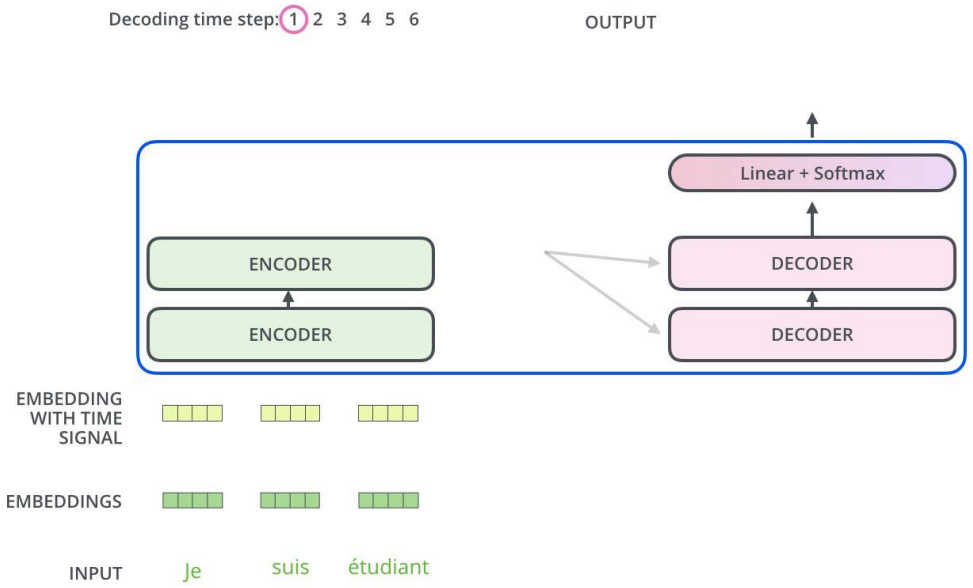


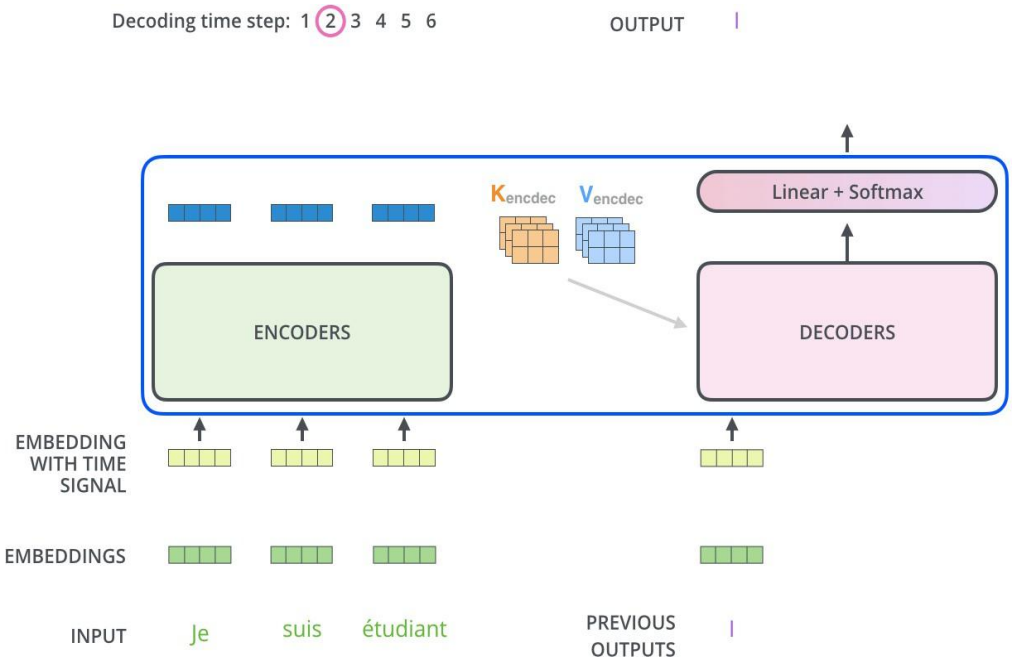
Image credit: https://jalammar.github.io/illustrated-gpt2/

Improving Language Understanding by Generative Pre-Training [Radford *et al.*, 2018]

1.30. Generation Repeats One Simple Loop

- 1. Read the current token prefix
- 2. Predict a distribution for the next token
- 3. Select one token
- 4. Append it and run again





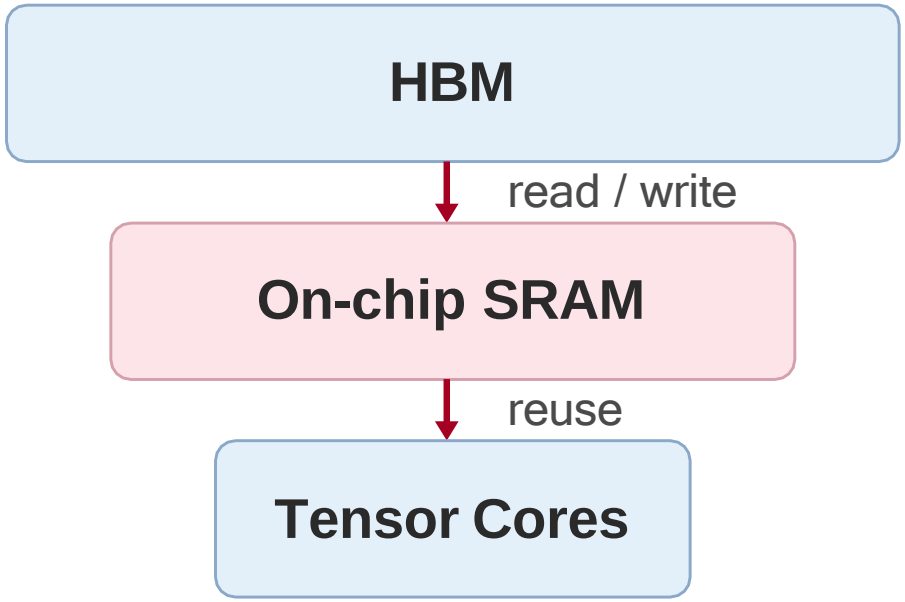
1.31. Transformer Components as Operators

Matrix multiplication	Reductions	Elementwise	Data movement
QKV projections Attention score/value products FFN layers	Softmax LayerNorm statistics	Activation Bias Residual add	Transpose / reshape Masking Reads and writes

Performance depends on how these operators use compute and memory.

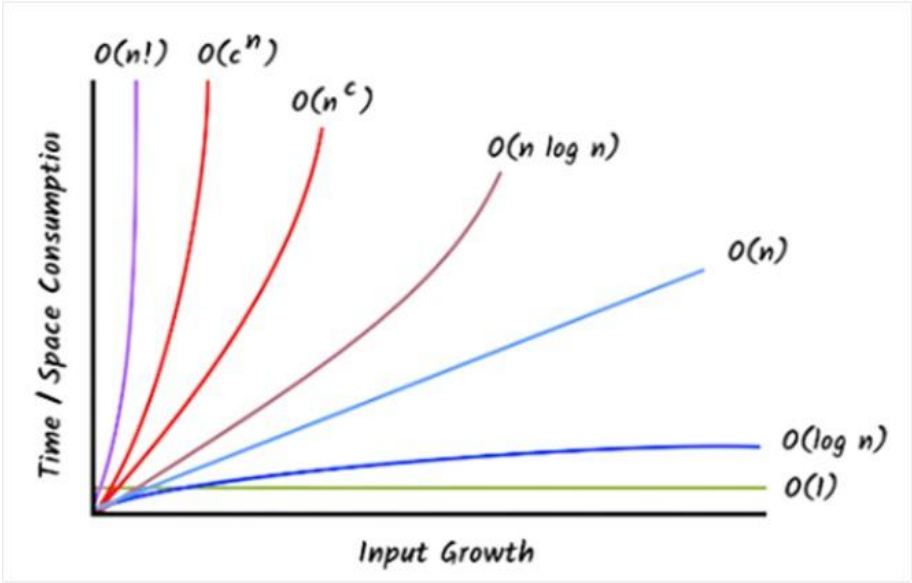
1.32. Data Movement Often Limits Attention

- Q, K, and V are read from GPU memory.
- Large intermediates are written out and read back.
- Moving bytes can dominate runtime.



1.33. Complexity of Attention (1/2)

- In transformer self-attention, we compute attention scores between all pairs of tokens:
  - Query Q is an (n,d) matrix (for n tokens, each with dimension d).
  - Key K is an (n,d) matrix.
  - We form an n × n attention score matrix QK^T.
  - We then multiply by Value V, which is (n,d) matrix.
- This requires on the order of n^2d operations
- As n grows, the quadratic term dominates.

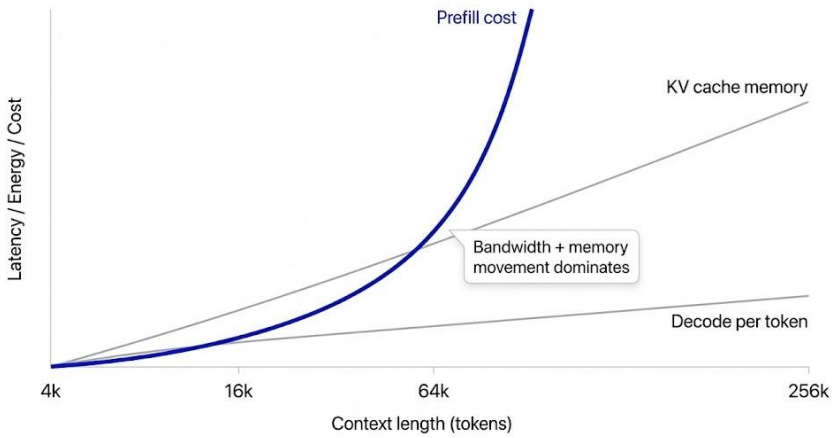


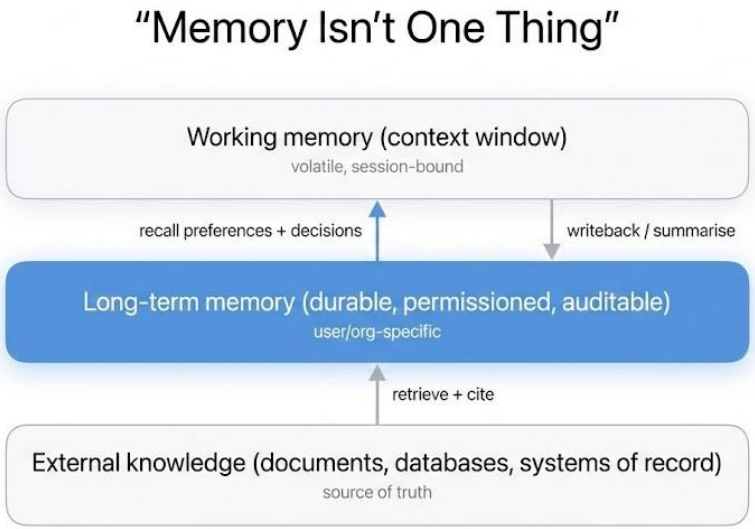
1.34. Complexity of Attention (2/2)

- 1. Compute Attention Scores
  - QK^T ⇒ (n,d) × (d,n) = (n,n)
  - The cost: about O(n·d·n) = **$O(n^2d)$**
  - This yields a **score matrix** (n,n) (basically all pairs of tokens).
- 2. Multiply by Values, V
  - **score matrix** (n,n) × V (n, d) = **(n, d)**
  - The cost again: O(n·n·d) = **$O(n^2d)$**
  - Hence **each forward pass** of a single-head attention block is about **$O(n^2d)$**.
- If it's **multi-head** with **h heads**, each head uses dimension **d/h** but we do it **h times**. Summed together, it's again $O(n^2d)$.

1.35. The Memory Wall Problem

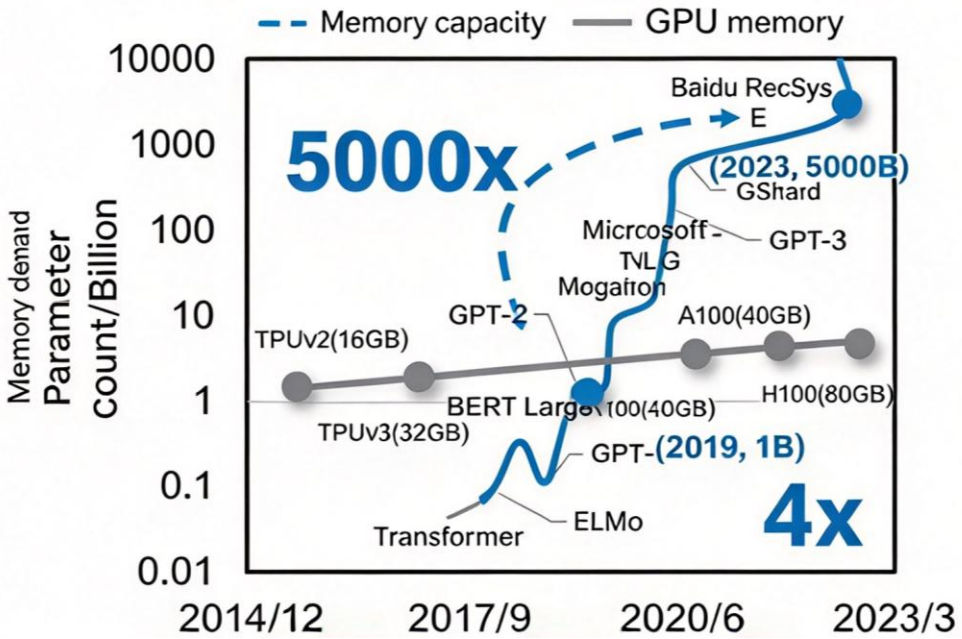
- Attention is expensive
  - Self-attention's flexibility comes at a steep cost:
  - its complexity grows quadratically with the length of the input sequence.
- Where transformers hit the wall
  - KV Cache Explosion
  - The Bandwidth Bottleneck
- Attention is all you need, until you need memory





1.36. The Memory Wall Problem

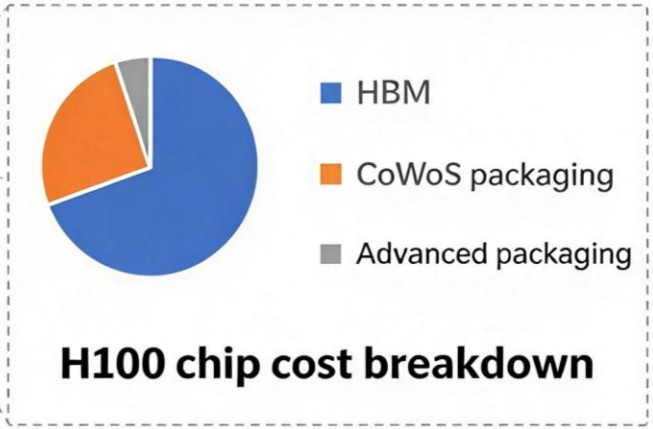
Memory demand has increased 5000x in recent years, while GPU physical size has only grown 4x



NVIDIA H100 GPU: up to 60% of cost comes from memory (HBM)*



NVIDIA H100

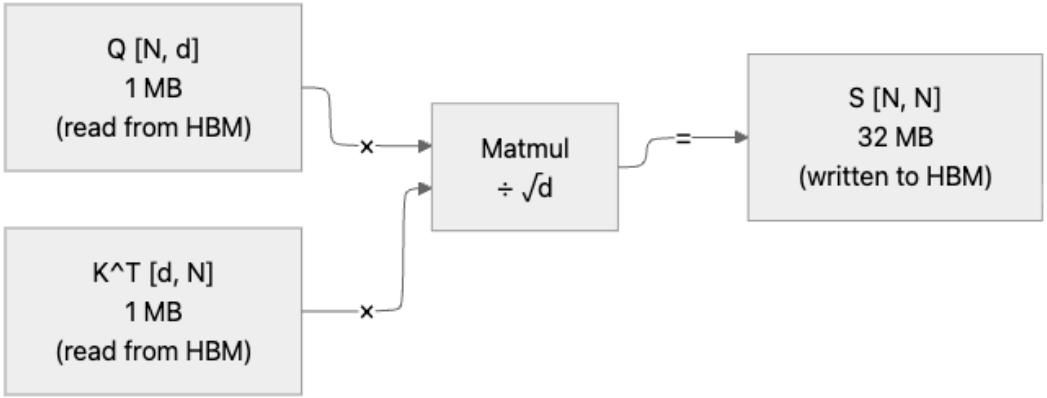


DeepSeek has extremely high demand for service storage:

- New KV cache added per day can reach 14 PB
- KV cache read Bandwich exceeed 1 TB/s

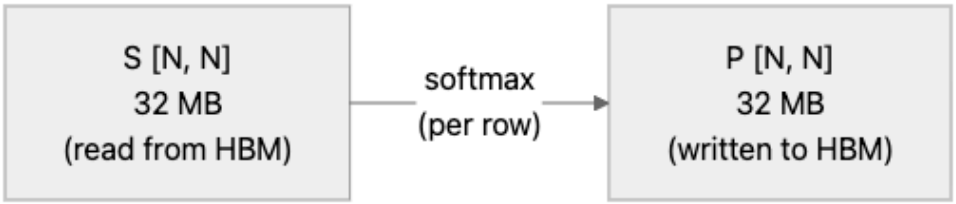
1.37. Data Flow Diagram of Standard Attention (1/2)

- Step 1: Computing the Attention Scores ($S = \frac{QK^T}{\sqrt{d}}$)



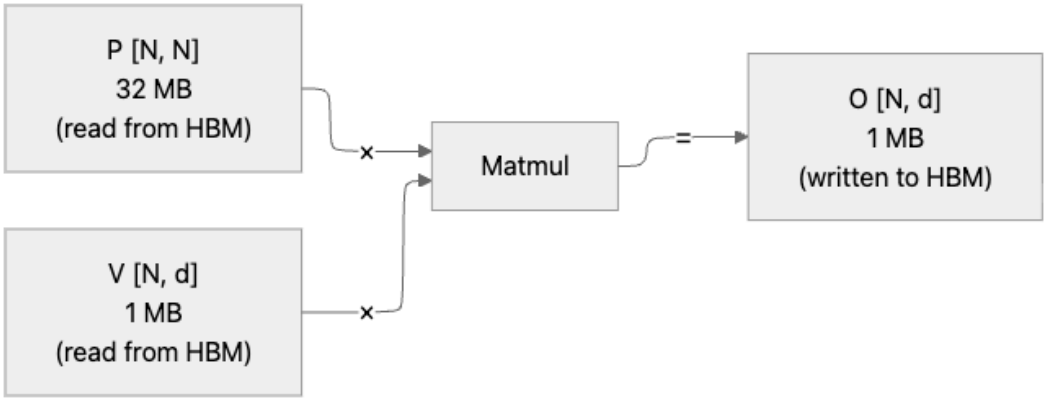
For example:
N = 4096, d = 128, FP16

- Step 2: Applying Softmax ($P = \mathrm{softmax}(S)$)



1.38. Data Flow Diagram of Standard Attention (2/2)

- Step 3: Computing the Output (O = PV)



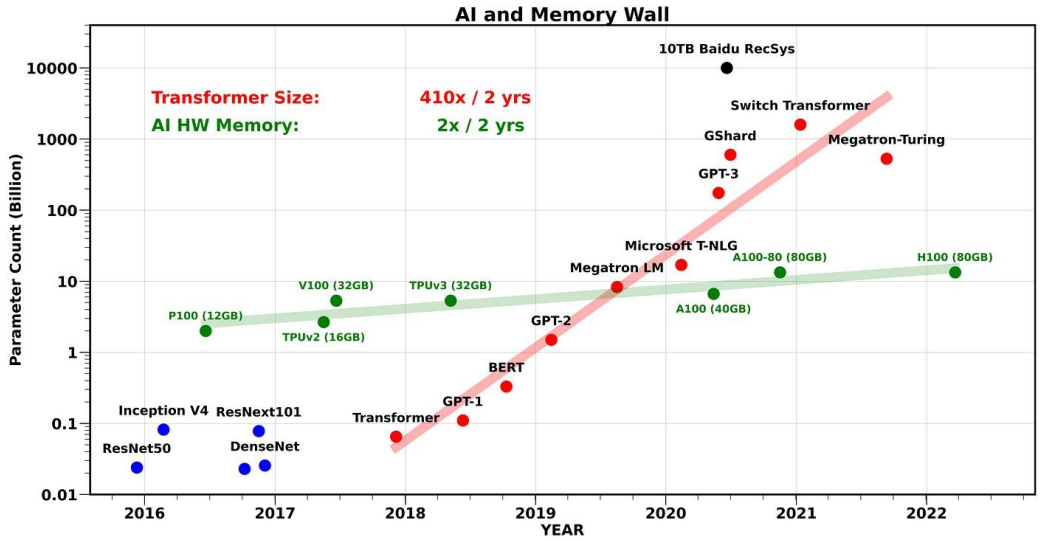
For example:
N = 4096, d = 128, FP16

- Total HBM Traffic:

Step	Reads from HBM	Writes to HBM	Subtotal
Step 1: S = QK^T / √d	Q (1 MB) + K (1 MB)	S (32 MB) ← N×N intermediate	34 MB
Step 2: P = softmax(S)	S (32 MB) ← N×N intermediate	P (32 MB) ← N×N intermediate	64 MB
Step 3: O = PV	P (32 MB) + V (1 MB)	O (1 MB)	34 MB
Total			**132 MB**

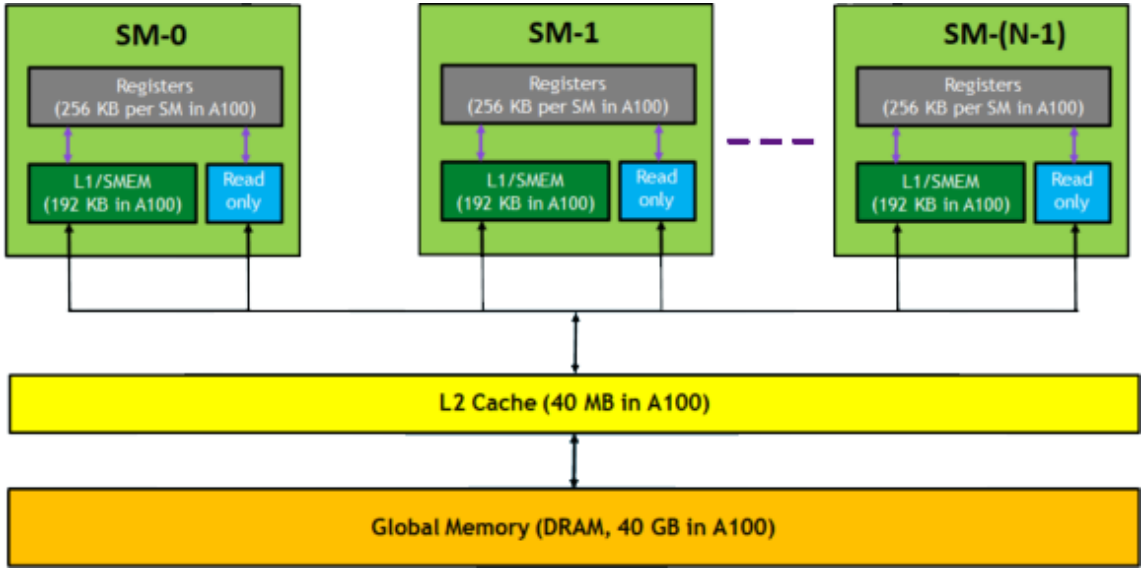
1.39. The “Memory Wall” of Standard Attention

- The Problem: Standard Attention complexity is $O(n^2)$.
- Pain Points: Memory consumption scales quadratically with sequence length n, leading to frequent Out-of-Memory errors during long-context processing.
- The Crucial Question: Do we truly need to materialize and store that massive N x N attention matrix?



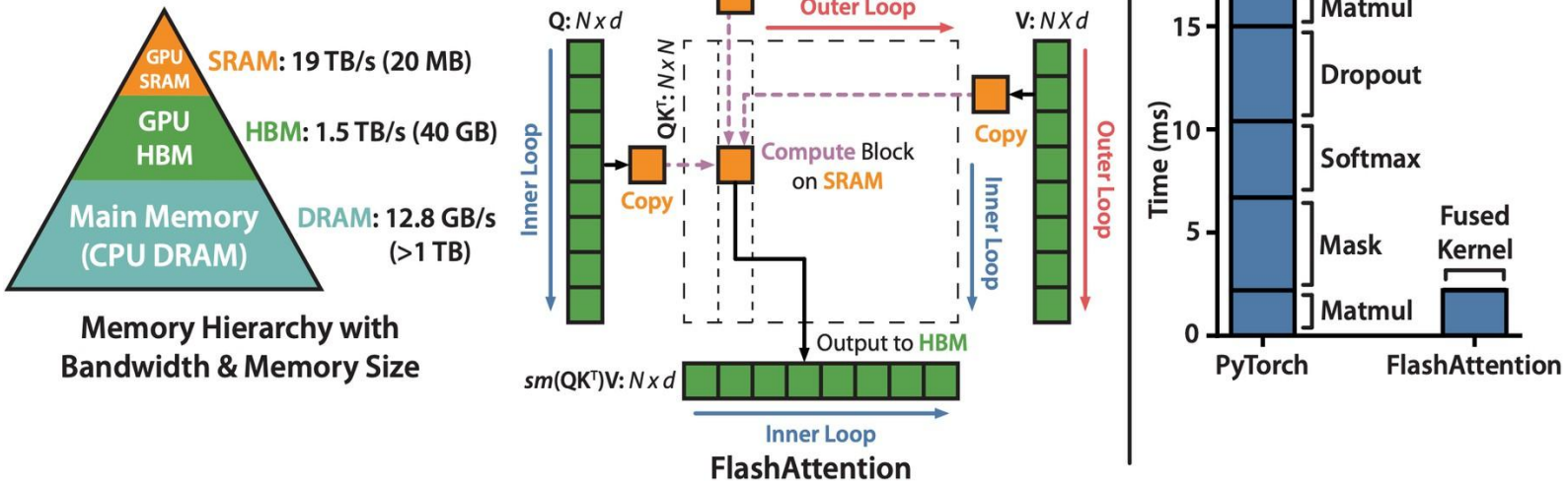
1.40. GPU Memory Hierarchy Explained

- Speed Disparity: HBM vs. SRAM (10x Factor)
  - For example: A100
  - GPU Memory / HBM2e: 40/80 GB 1.5-2 TB/s
  - L2 Cache: 40 MB 5 TB/s
  - L1 Cache / Shared Memory: 192 KB/SM 19 TB/s



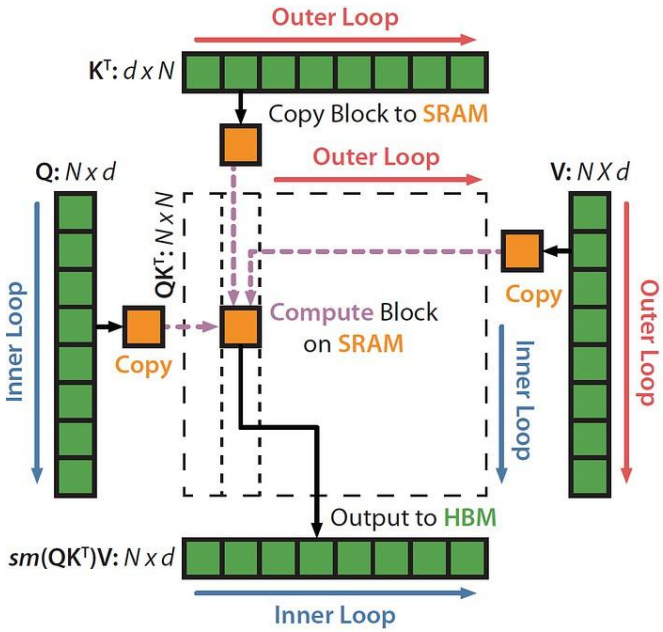
1.41. Core Idea of FlashAttention: Less Data Movement

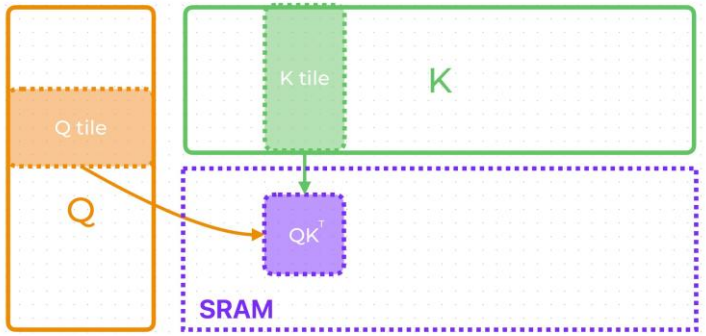
- Less Movement, More Speed
  - Design Philosophy: Optimize Data Movement, Not Just Computation
- Two Technical Pillars:
  - Tiling: Breaking down large tasks into smaller blocks that fit the SRAM capacity.
  - Recomputation: Instead of storing intermediate activation values during the forward pass, they are recalculated during the backward pass to save memory space.



1.42. Tiling Concept

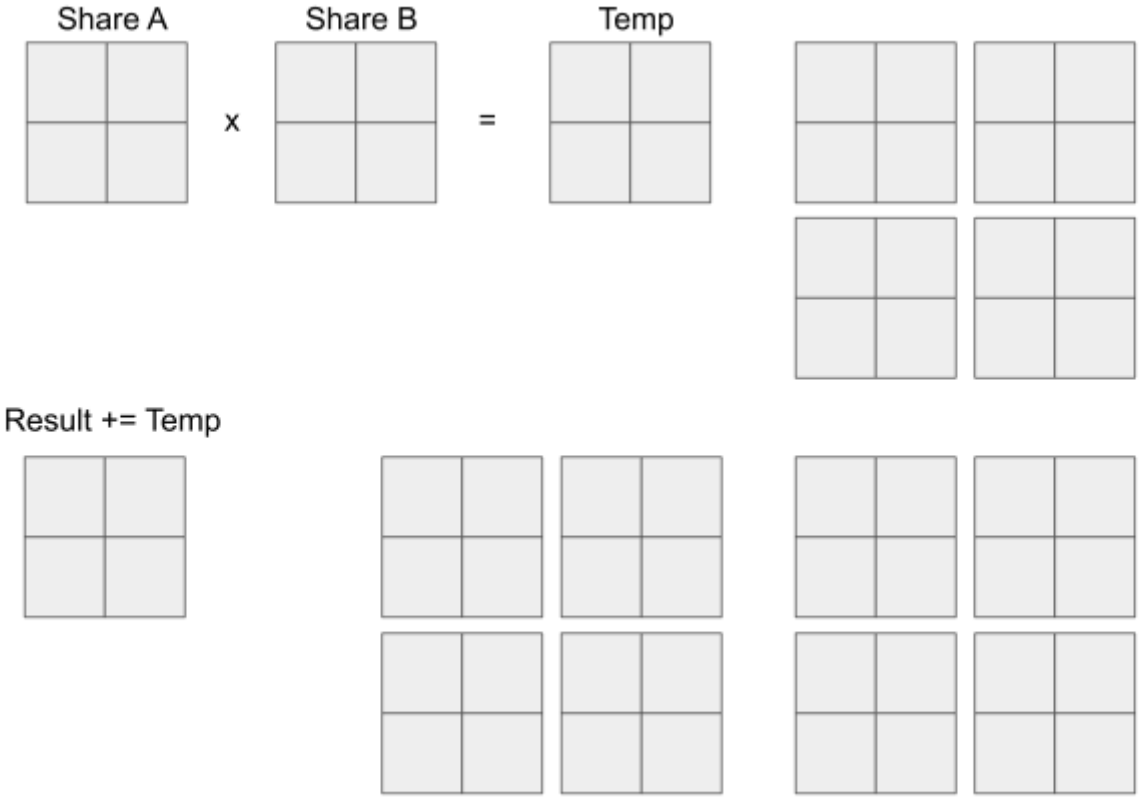
- Why Tiling?
  - GPU SRAM (e.g., L1/Shared Memory in A100) is extremely fast but has very limited capacity (typically a few hundred KB).
  - Traditional N x N matrices (e.g., when N=4096) far exceed SRAM capacity and must be partitioned into “tiles” that can fit into the cache.
- How It Works
  - Input matrices Q, K, and V are partitioned into blocks (tiles). Using nested loops, these blocks are loaded from HBM to SRAM piece by piece for local computation.
  - Performance Gain: Partial dot-products and Softmax accumulations are completed entirely within SRAM. This eliminates the need to write N x N intermediate results back to the slow HBM.





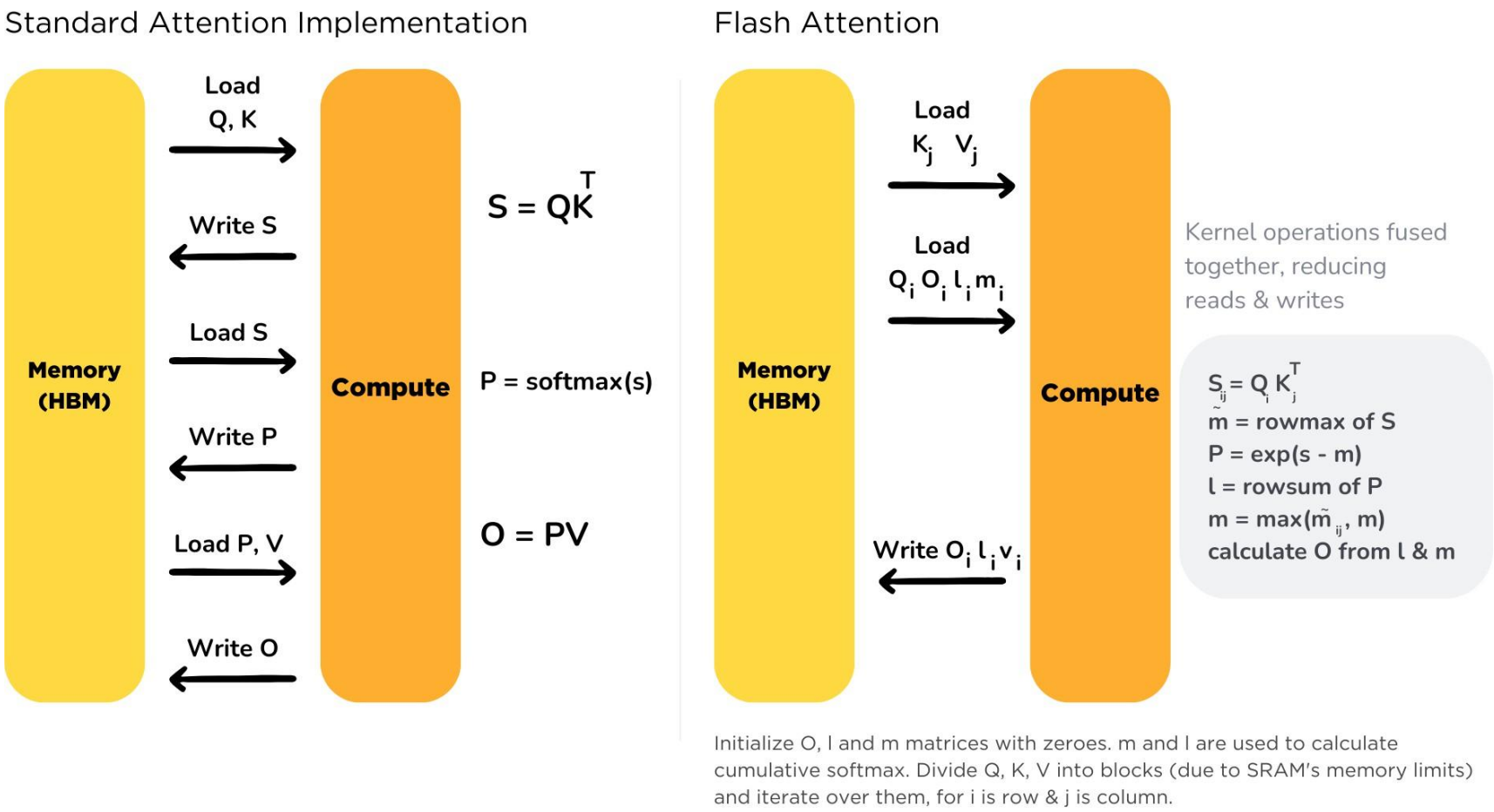
1.43. Dynamic Demo

- The process of loading tiled blocks.



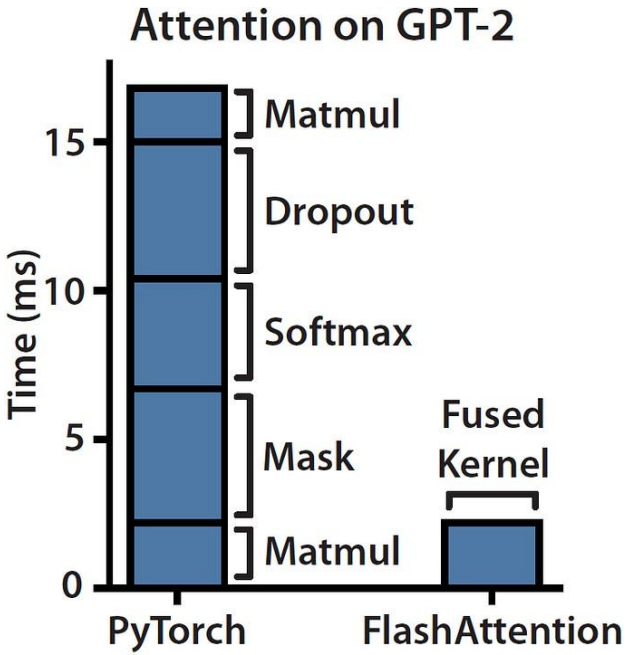
1.44. Memory Access Comparison

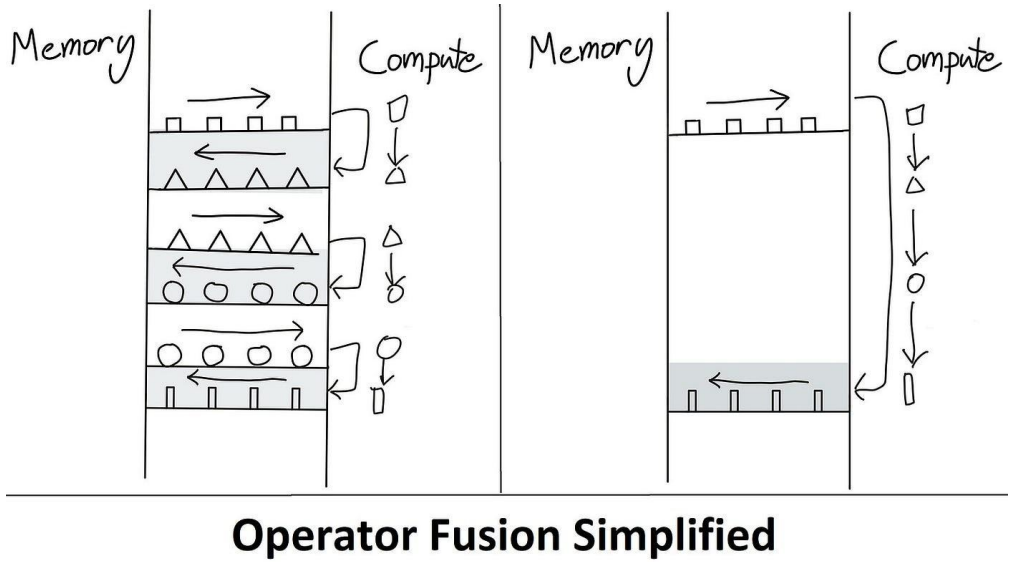
- Standard Attention vs. FlashAttention



1.45. Kernel Fusion

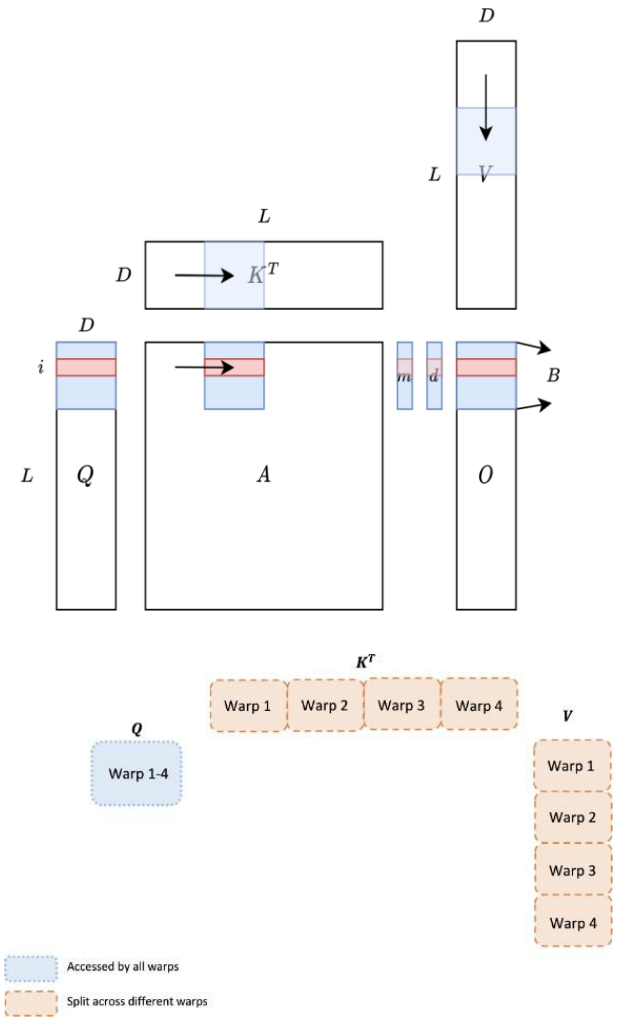
- Standard attention launches separate kernels
  - Each launch clears the SRAM and writes results back to HBM, causing massive I/O waste.
- FlashAttention fuses attention operations into one optimized kernel
  - SRAM-Centric Execution: Perform tiled loading, dot-product, online softmax, and weighted sum within a single loop.





1.46. Forward Pass Implementation

- How Tiling enables computation without storing the N x N matrix.
  - Outer Loop (Loop over K, V blocks): Divide K and V matrices into column blocks and load them into SRAM.
  - Inner Loop (Loop over Q blocks): Divide Q into row blocks. For each Qi, perform the computation.
  - On-chip Computation: Compute local attention scores in SRAM and immediately apply weighted summation with V.
  - Write-back: Only the final output matrix O (size N x d) is written back to HBM, completely bypassing the $O(n^2)$ intermediate storage.

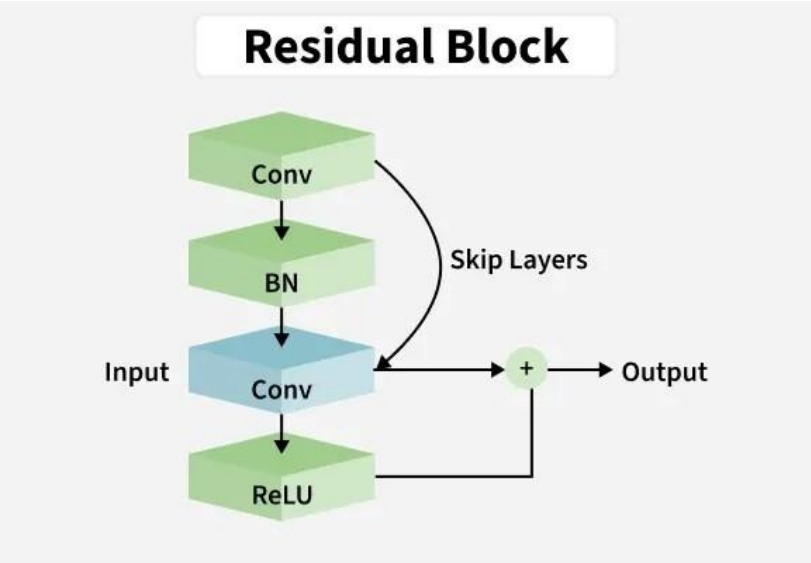


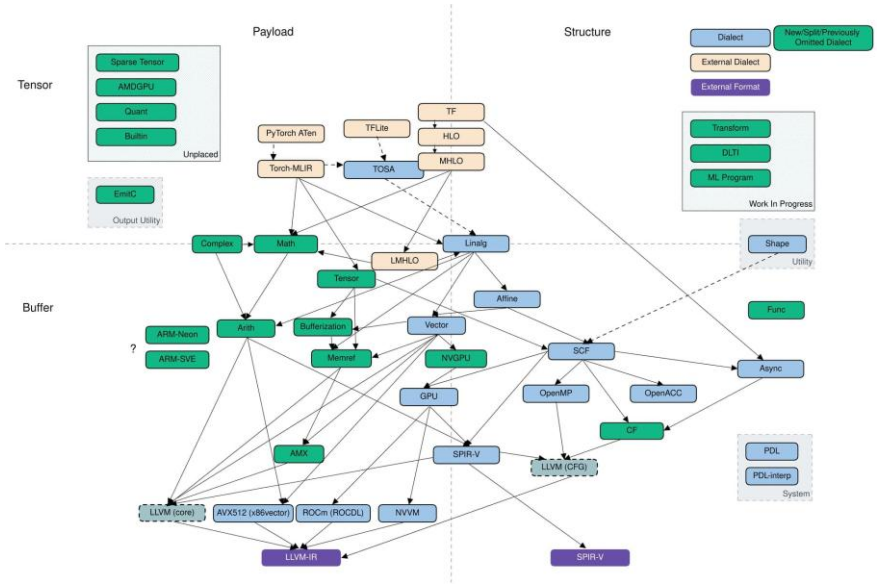
2. Operator Fusion and Case Study

- Lifecycle of Operators
- Kernel Launch Optimization: CUDA Graph
- Data Transfer Between Operators
- Operator Fusion
- Advantages of Operator Fusion
- Problems of Operator Fusion
- Example: RMSNorm + Residual
- fused_add_rms_norm_kernel

2.1. Lifecycle of Operators

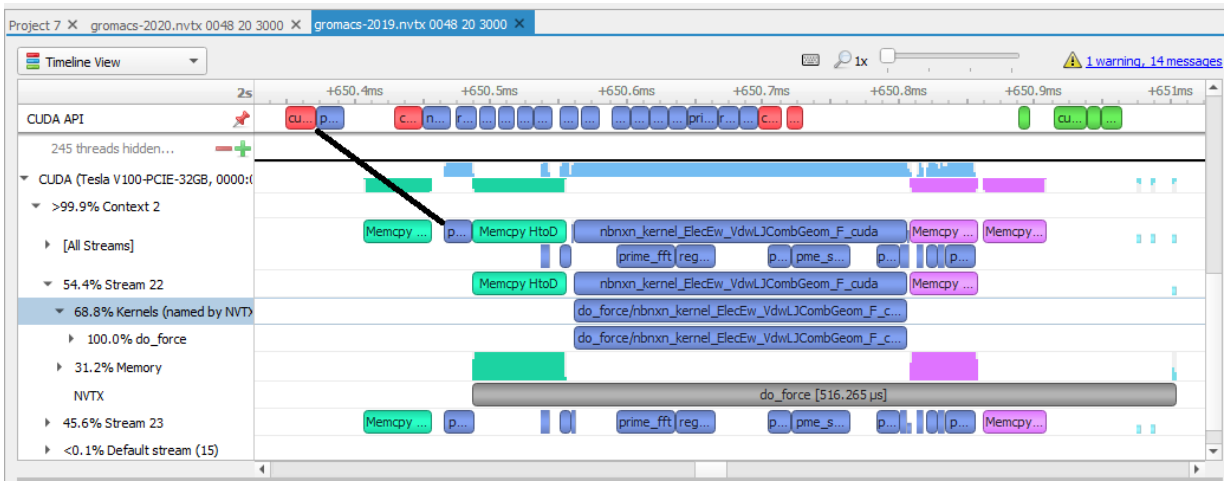
- Framework layer:
  - Data graph normalization: Break down high-level composite operators in deep neural networks (e.g., TransformerBlock) into fine-grained primitives (MatMul, Softmax, LayerNorm), and establish standardized IR frameworks (e.g., MLIR, XLA HLO).
  - Calculate the tensor lifecycle through data stream analysis and determine the parallel strategy
  - Convert the optimized compute graph Lowering into device-specific code and insert necessary synchronization primitives





2.2. Lifecycle of Operators

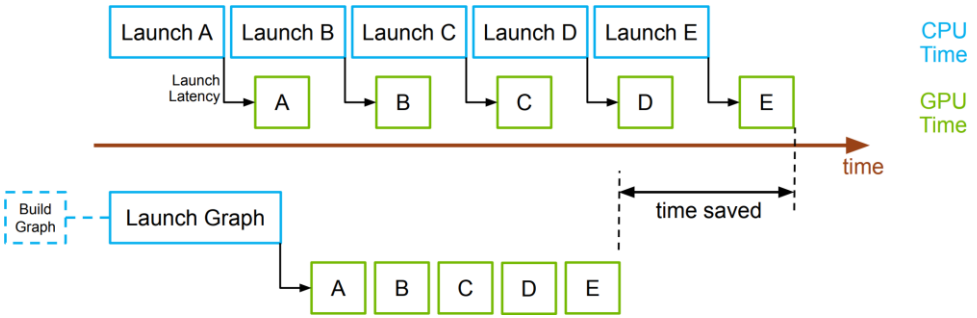
- Runtime layer
  - Prepare parameters for each operator (device pointer, shared memory size, Stream ID, and other context)
  - Initiate Kernel Launch
- Hardware layer
  - GPU reads data from Global Memory → compute unit processes → writes back to Global Memory



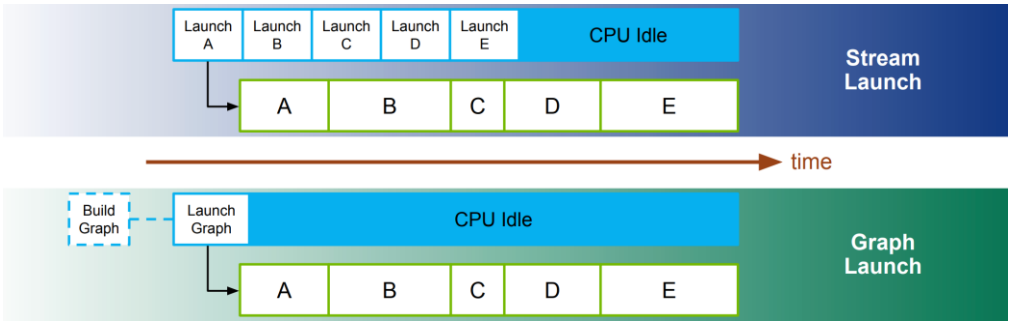
2.3. Kernel Launch Optimization: CUDA Graph

- Record a series of GPU operations (Kernels, Memcpys) as a static topology diagram
  - Capture the execution sequence, parameters, video memory addresses, and scheduling paths of all kernels during the phase.
  - During the replay phase, the CPU only needs to issue a single ultra-lightweight instruction to trigger the entire graphs automatic transfer on the GPU.

- minimal GPU kernel

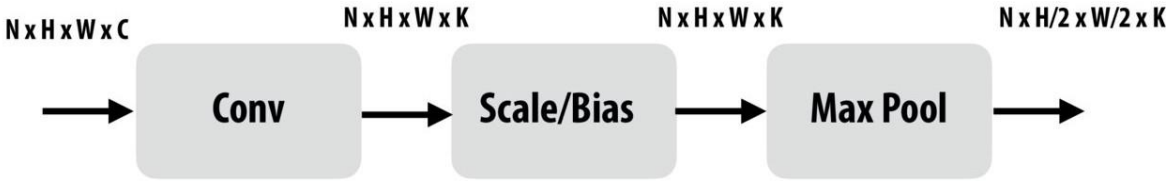


- general-purpose GPU kernel



2.4. Data Transfer Between Operators

- Default mode: Conv results must be written back to global video memory, then Scale reads them out

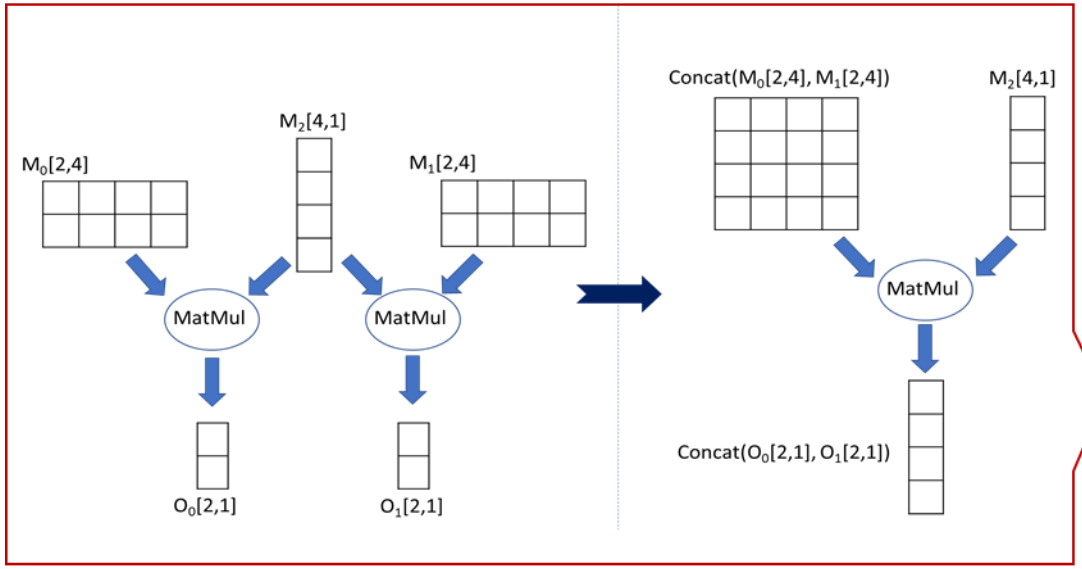


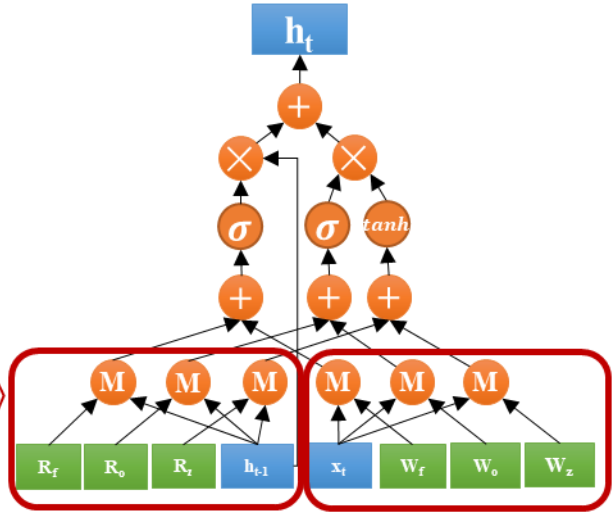
- Operator fusion: Element-wise operations can seamlessly integrate with other operators, significantly reducing both memory access and kernel startup overhead.



2.5. Operator Fusion

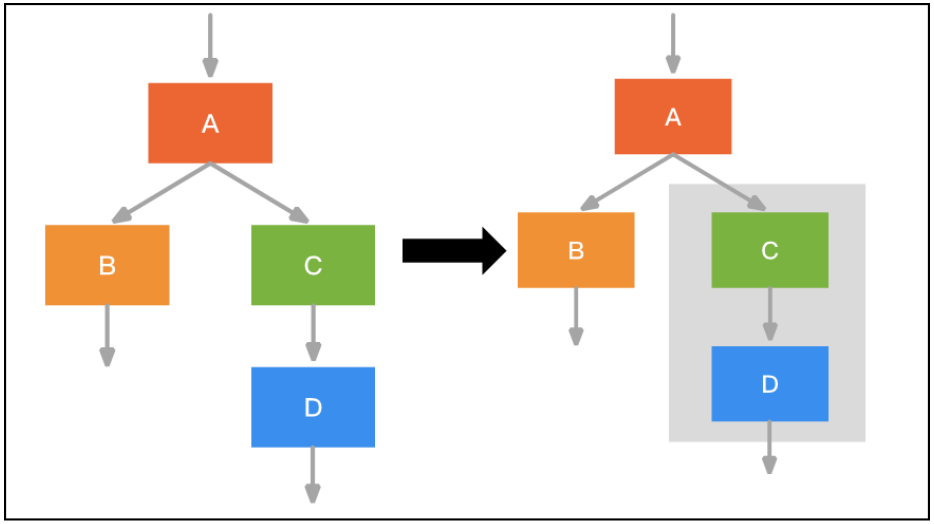
- Definition: A technique that merges multiple independent operators (or kernels) in a computational graph into a single operator for execution.
- Typical scenarios in deep learning:
  - Vertical Fusion: Conv + Bias + ReLU
  - Horizontal Fusion: Merge multiple GEMMs with the same input but different parameters
  - Reduction Fusion: Mean + Var + Scale in LayerNorm

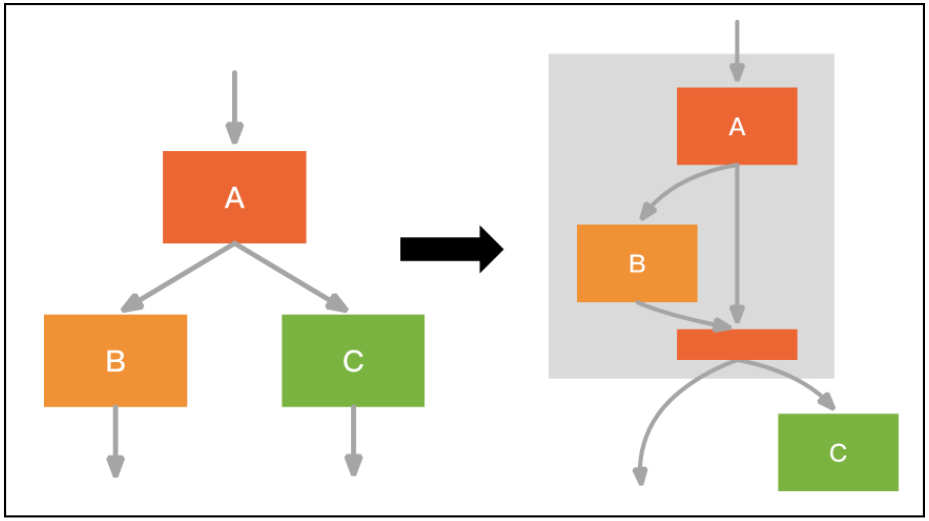


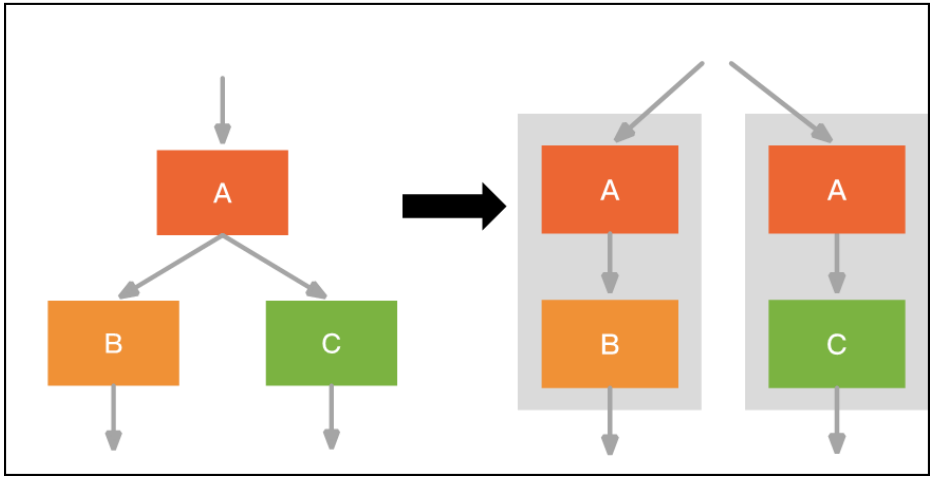


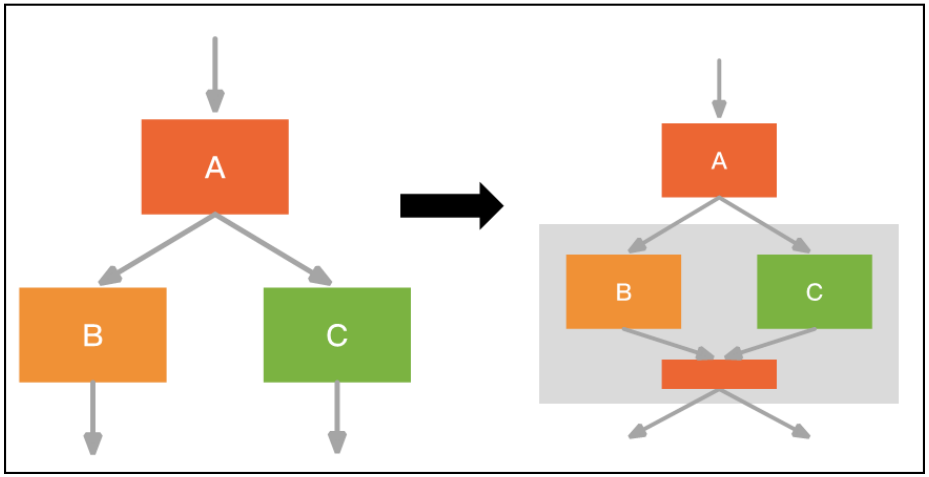
Data-flow graph of a GRU cell

2.6. Operator Fusion



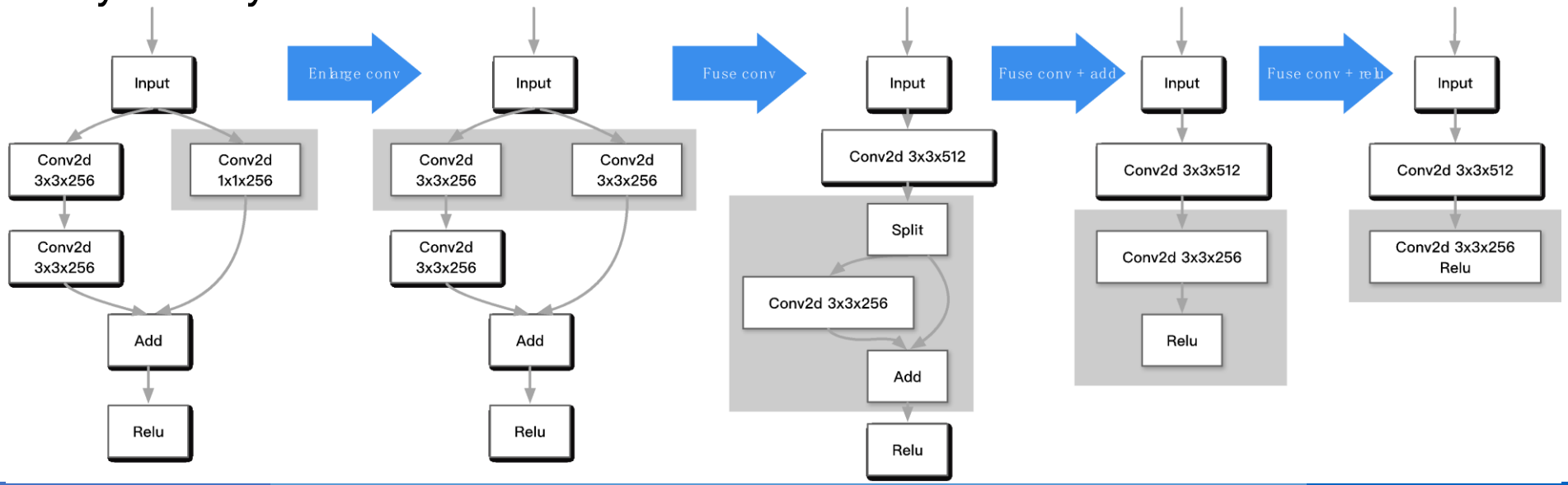






2.7. Advantages of Operator Fusion

- Reduce memory bandwidth usage: Eliminate read and write operations on global memory for intermediate results. Data remains in registers or shared memory.
- Reduce Kernel Launch overhead: Minimize the number of handshakes of CPU/GPU
- Enhance cache utilization: Data is continuously accessed in SRAM for higher efficiency
- Increase computational density: Boost the instruction stream per task to better mask memory latency

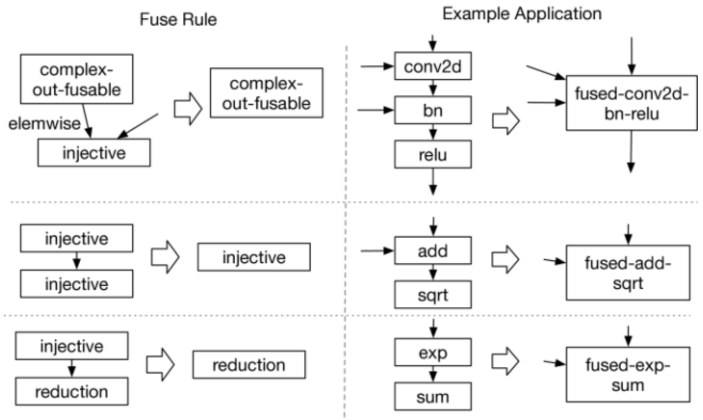


2.8. Problems of Operator Fusion

- Core trade-off: register pressure
  - The total number of registers in a GPU SM is fixed (for example, A100 has 256KB per SM).
  - Single-threading requires more intermediate states → Single-threading registers ↑ → Each thread block occupies registers ↑ → Concurrent thread blocks decrease → Reduced active Warp count in SM (Occupancy decreases)
- Instruction cache pressure:
  - The merged kernel may exceed the hardware instruction cache limit.
- Finding the sweet spot between saving memory bandwidth and maintaining sufficient computational parallelism



TVM end-to-end deep learning compiler



2.9. Example: RMSNorm + Residual

```
def naive_add_rms_norm(x, residual, weight, eps=1e-6):
    # Residual addition
    updated = x + residual
    updated_residual = updated

    # Mean square and reciprocal RMS
    variance = (updated * updated).mean(dim=-1)
    rrms = torch.rsqrt(variance + eps)

    # Normalize and apply the learned weight
    out = updated * rrms[:, None] * weight
    return out, updated_residual
```

RMSNorm

$$\mathrm{RMSNorm}(\mathbf{x}) = \mathbf{y} = \frac{\mathbf{x}}{\mathrm{RMS}(\mathbf{x}) + \epsilon} \odot \mathbf{w}$$

$$\mathrm{RMS}(\mathbf{x}) = \sqrt{\frac{1}{d}\sum_{i=1}^{d} x_i^2}$$

- Performance bottleneck: Total reads 5MN + 2M + N elements, writes 2MN + 2M elements. Frequent global memory reads/writes cause significant bandwidth waste.

2.10. Example: RMSNorm + Residual

- Basic Idea of Operator Fusion
  - Load: All threads collaborate in parallel to read data from Global Memory and write it to Shared Memory
  - Sync & Reduce: Perform multiple iterations within Shared Memory, which is fully executed at high speed on-chip without consuming video memory bandwidth.
    - CUDA: Synchronize with __syncthreads() at each step
    - Triton: Automatic
  - Write: Only the final result is written back to Global Memory.

2.11. fused_add_rms_norm_kernel

On-chip reuse: keep the loaded row available through normalization.

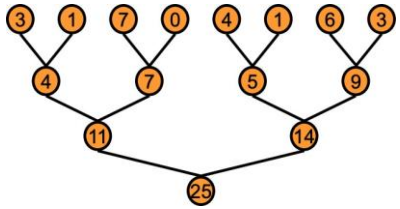
```
@triton.jit
def _fused_add_rms_norm_kernel(
    x_ptr, residual_ptr, out_ptr, updated_residual_ptr,
    weight_ptr, n_cols: tl.constexpr, eps, BLOCK_SIZE: tl.constexpr,
):
    # Compute row pointers and a boundary mask
    pid = tl.program_id(0)
    cols = tl.arange(0, BLOCK_SIZE)
    mask = cols < n_cols
    row_offsets = pid * n_cols + cols
    # Load the inputs
    x = tl.load(x_ptr + row_offsets, mask=mask, other=0.0).to(tl.float32)
    residual = tl.load(residual_ptr + row_offsets, mask=mask, other=0.0).to(tl.float32)
    weight = tl.load(weight_ptr + cols, mask=mask, other=0.0).to(tl.float32)
    # Add, reduce, normalize, and scale
    updated = x + residual
    tl.store(updated_residual_ptr + row_offsets, updated, mask=mask)
    variance = tl.sum(updated * updated, axis=0) / n_cols
    rrms = 1.0 / tl.sqrt(variance + eps)
    out = updated * rrms * weight
    # Write the normalized output
    tl.store(out_ptr + row_offsets, out, mask=mask)
```

2.12. fused_add_rms_norm_kernel

Operator fusion: residual addition, RMS reduction, normalization, and scaling.

```
@triton.jit
def _fused_add_rms_norm_kernel(
    x_ptr, residual_ptr, out_ptr, updated_residual_ptr,
    weight_ptr, n_cols: tl.constexpr, eps, BLOCK_SIZE: tl.constexpr,
):
    # Compute row pointers and a boundary mask
    pid = tl.program_id(0)
    cols = tl.arange(0, BLOCK_SIZE)
    mask = cols < n_cols
    row_offsets = pid * n_cols + cols
    # Load the inputs
    x = tl.load(x_ptr + row_offsets, mask=mask, other=0.0).to(tl.float32)
    residual = tl.load(residual_ptr + row_offsets, mask=mask, other=0.0).to(tl.float32)
    weight = tl.load(weight_ptr + cols, mask=mask, other=0.0).to(tl.float32)
    # Add, reduce, normalize, and scale
    updated = x + residual
    tl.store(updated_residual_ptr + row_offsets, updated, mask=mask)
    variance = tl.sum(updated * updated, axis=0) / n_cols
    rrms = 1.0 / tl.sqrt(variance + eps)
    out = updated * rrms * weight
    # Write the normalized output
    tl.store(out_ptr + row_offsets, out, mask=mask)
```

Parallel tree reduction



2.13. RMSNorm + Residual Host

```
def _block_size(n_cols):
    # Smallest power of two covering a full row
    return triton.next_power_of_2(n_cols)

def fused_add_rms_norm(x, residual, weight, eps=1e-6):
    # Allocate device memory for both outputs
    out = torch.empty_like(x)
    updated_residual = torch.empty_like(x)
    n_rows, n_cols = x.shape

    # Specialize the kernel for this block size
    block_size = _block_size(n_cols)

    # Launch one program per row:
    # residual add, RMS, normalization, weight scaling
    _fused_add_rms_norm_kernel[(n_rows,)](
        x, residual, out, updated_residual, weight,
        n_cols, eps, BLOCK_SIZE=block_size,
    )
    return out, updated_residual
```

Static specialization

Use the smallest power of two that covers the row.

BLOCK_SIZE is a compile-time constant.

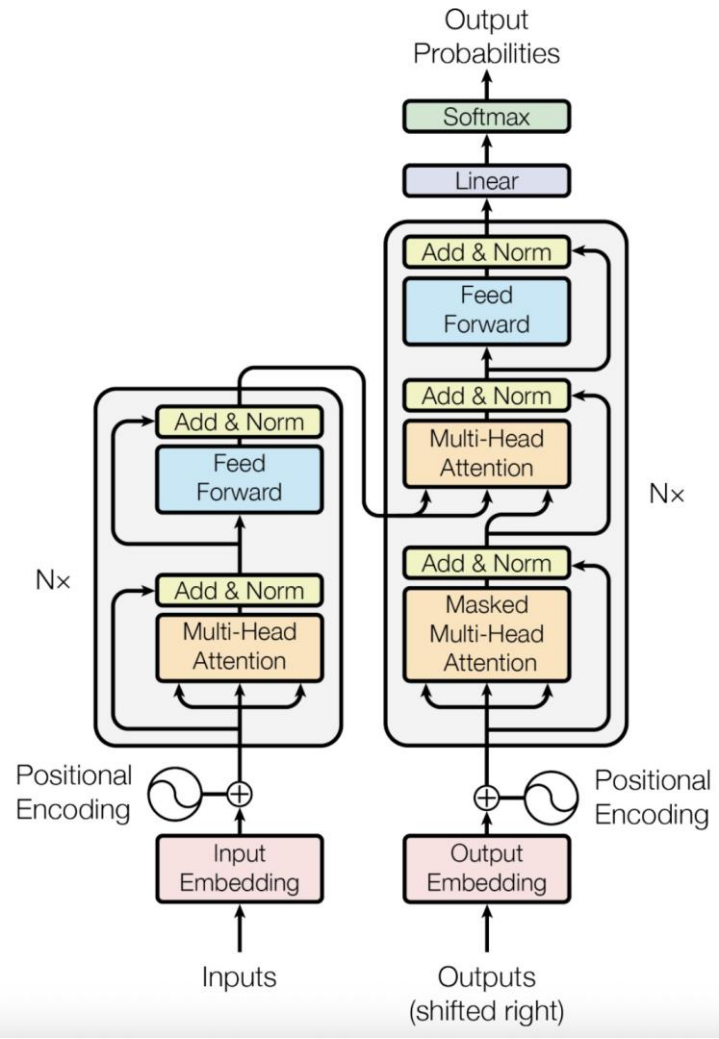
The launch grid has one program per matrix row.

3. Architectural Design of Operator Libraries

- Principles
- Dispatcher
- DSL
- Case Study

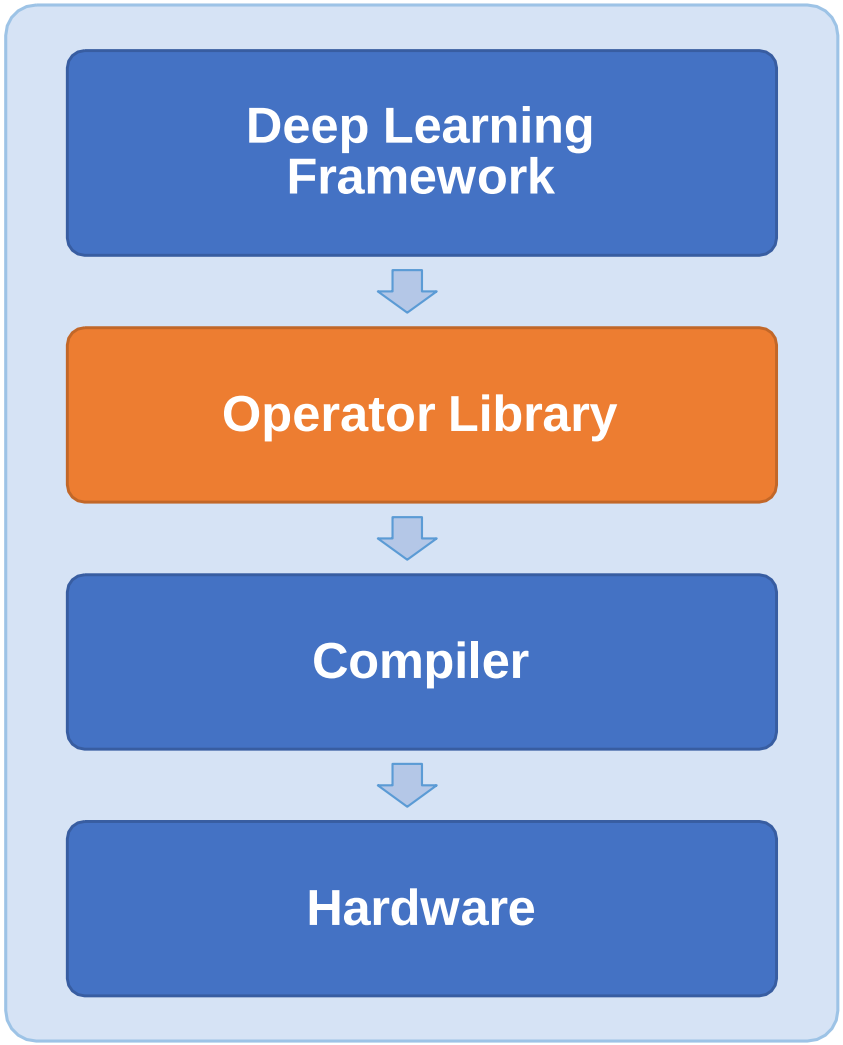
3.1. Operator

- Broadly speaking, applying any specific operation to a given function can be defined as an Operator.
- Deep learning algorithms are constructed from individual computational units known as Operators.
- Examples: Add, MatMul, LayerNorm



3.2. Operator Library

- A collection of highly optimized, reusable computational building blocks (Operators) for neural networks.
- Core Roles:
  - The Bridge – Connects high-level AI frameworks with low-level bare-metal hardware.
  - Performance Extractor – Maximizes hardware utilization via expert-level kernel tuning.
  - Hardware Abstraction – Shields algorithm developers from complex low-level programming.



3.3. Design Principles - Usability

- Seamless Integration with High-Level Frameworks
  - Register the operator library directly into the framework's internal dispatch mechanism to allow existing models to benefit from acceleration with zero code modifications.
- Automatic Hardware Discovery
  - Implement runtime detection of the execution environment to automatically load the optimal backend without requiring manual user configuration.
- Uniform Interface for Diverse Backends
  - Define a standardized abstraction layer so that the same high-level API remains consistent across a wide variety of hardware architectures
- Multi-Level Abstraction Architecture
  - Offer a tiered abstraction system to meet diverse needs: high-level for rapid prototyping, mid-level for tuning parallelism and tiling, and low-level for expert optimization of hardware specifics like registers and warps

3.4. Design Principles - Performance

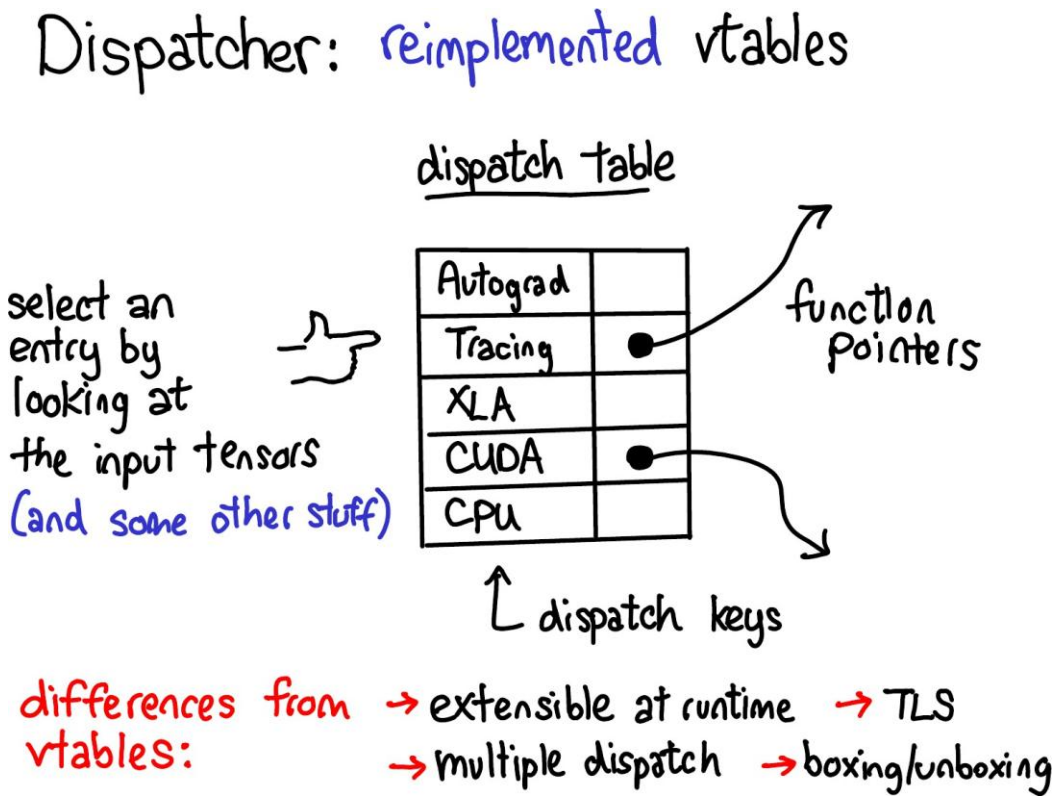
- Persistent Caching and Pre-Tuning
  - Store optimized computational configurations and pre-warm kernels to eliminate initialization delays and ensure peak performance from the very first execution.
- Hardware-Aware Configuration Mapping
  - Utilize a configuration system that abstracts hardware-specific constraints, such as memory hierarchy and compute unit limits, into the code generation process.
- Adaptive Tiling
  - Decomposes operators into optimal compute tiles based on input dimensions, minimizing padding overhead and maximizing memory bandwidth utilization across variable sequence lengths.

3.5. Trade-offs

- Dynamic Shape vs. Hardware Alignment
  - Arbitrary input sizes often limit hardware-aligned peak performance
  - Solution: Adaptive Tiling
- Intuitive Layout vs. Optimal Memory Access
  - Standard data formats often conflict with hardware-optimized memory patterns
  - Solution: Layout Transformation
- Black-box API vs. Fine-grained Tuning
  - Simplified interfaces hide the tuning knobs needed for expert optimization
  - Solution: Heuristic Mapping
- A dispatch mechanism is required to manage these automated solutions and bridge the gap between user intent and hardware execution.

3.6. Dispatcher

- A dispatcher is a component that routes an operation to the appropriate implementation or backend based on runtime criteria such as device type, data properties, or configuration policies.



3.7. Simplified Dispatch Workflow

1. Metadata Collection
   - Extract metadata such as device, data type, and layout from input tensors
2. Key Computation
   - Combine multiple attributes to generate a unique Dispatch Key
3. Table Lookup
   - Search the global Dispatch Table for the corresponding kernel function pointer
4. Execution
   - Invoke the specific backend implementation to perform the computation

3.8. Advanced Dispatching Mechanisms

- Sequential Re-dispatching
  - A single operator call can trigger a chain of handlers by masking processed keys and re-invoking the dispatcher to find the next implementation.
- Heuristic Implementation Selection
  - Profile workload metrics (e.g., shapes) and hardware attributes (e.g., SM counts) to adaptively select optimal algorithm paths and execution parameters.
- Dynamic Kernel Instantiation
  - Specialized kernels can be generated and cached on-the-fly to handle various task dimensions or broadcasting patterns without manual coding.

3.9. Use DSL to Implement Operator Libraries

- Developers describe “what to do,” while the compiler determines “how to do it.”
- A total separation between operator logic and physical implementation
- Core Objective: Make operator development simpler and more efficient while ensuring cross-platform portability
- Examples: Triton, TileLang, Gluon

3.10. DSL for Operator Library - Triton

- Python-Based Tile Programming
- Compiler-Driven Optimization
- Hardware-Agnostic Abstraction



```
@triton.jit
def add_kernel(
    x_ptr,          # First input vector
    y_ptr,          # Second input vector
    output_ptr,     # Output vector
    n_elements,     # Total number of elements
    BLOCK_SIZE: tl.constexpr,
):
    # Each program handles one block
    pid = tl.program_id(axis=0)
    block_start = pid * BLOCK_SIZE
    offsets = block_start + tl.arange(0, BLOCK_SIZE)

    # Guard accesses beyond the final element
    mask = offsets < n_elements
    x = tl.load(x_ptr + offsets, mask=mask)
    y = tl.load(y_ptr + offsets, mask=mask)
    output = x + y

    # Write the result to global memory
    tl.store(output_ptr + offsets, output, mask=mask)
```

3.11. Case Study: Operator Definition

Defining the operator schema

Separate the operator interface from its hardware implementation.

```
# Keep Library objects alive to preserve their registrations.
_definition = torch.library.Library("lab", "DEF")
_definition.define(
    "fused_add_rms_norm(Tensor x, Tensor residual, Tensor weight, "
    "float eps=1e-6) -> (Tensor, Tensor)"
)

def fused_add_rms_norm_op(x, residual, weight, eps=1e-6):
    """Convenience wrapper for the registered operator."""
    return torch.ops.lab.fused_add_rms_norm(x, residual, weight, eps)
```

3.12. Implementing the Dispatcher

CUDA dispatch: the performance path

```
def _cuda_impl(x, residual, weight, eps=1e-6):
    """Invoke the Triton forward kernel."""
    return fused_add_rms_norm(x, residual, weight, eps)

_cuda = torch.library.Library("lab", "IMPL", "CUDA")
_cuda.impl("fused_add_rms_norm", _cuda_impl)
```

Meta dispatch: the compilation path

```
def _meta_impl(x, residual, weight, eps=1e-6):
    """Return output metadata for graph capture."""
    return torch.empty_like(x), torch.empty_like(residual)

_meta = torch.library.Library("lab", "IMPL", "Meta")
_meta.impl("fused_add_rms_norm", _meta_impl)
```

3.13. Backpropagation

计算误差对权重 w_{ij} 的偏导数是两次使用链式法则得到的：

$$\frac{\partial E}{\partial w_{ij}} = \frac{\partial E}{\partial o_j}\frac{\partial o_j}{\partial \mathrm{net_j}}\frac{\partial \mathrm{net_j}}{\partial w_{ij}}$$

在右边的最后一项中，只有加权和 $\mathrm{net_j}$ 取决于 w_{ij}，因此

$$\frac{\partial \mathrm{net_j}}{\partial w_{ij}} = \frac{\partial}{\partial w_{ij}}\left(\sum_{k=1}^{n} w_{kj}o_k\right) = o_i.$$

神经元 j 的输出对其输入的导数就是激活函数的偏导数（这里假定使用逻辑函数）：

$$\frac{\partial o_j}{\partial \mathrm{net_j}} = \frac{\partial}{\partial \mathrm{net_j}}\varphi(\mathrm{net_j}) = \varphi(\mathrm{net_j})(1-\varphi(\mathrm{net_j}))$$

这就是为什么反向传播需要的激活函数是可微的。

如果神经元在输出层中，因为此时 $o_j = y$ 以及

$$\frac{\partial E}{\partial o_j} = \frac{\partial E}{\partial y} = \frac{\partial}{\partial y}\frac{1}{2}(t-y)^2 = y - t$$

所以第一项可以直接算出。

但如果 j 是网络中任一内层，求 E 关于 o_j 的导数就不太简单了。

考虑 E 为接受来自神经元 j 的输入的所有神经元 $L = u, v, \ldots, w$ 的输入的函数，

$$\frac{\partial E(o_j)}{\partial o_j} = \frac{\partial E(\mathrm{net}_u, \mathrm{net}_v, \ldots, \mathrm{net}_w)}{\partial o_j}$$

并关于 o_j 取全微分，可以得到该导数的一个递归表达式：

$$\frac{\partial E}{\partial o_j} = \sum_{l\in L}\left(\frac{\partial E}{\partial \mathrm{net}_l}\frac{\partial \mathrm{net}_l}{\partial o_j}\right) = \sum_{l\in L}\left(\frac{\partial E}{\partial o_l}\frac{\partial o_l}{\partial \mathrm{net}_l}\frac{\partial \mathrm{net}_l}{\partial o_j}\right) = \sum_{l\in L}\left(\frac{\partial E}{\partial o_l}\frac{\partial o_l}{\partial \mathrm{net}_l}w_{jl}\right)$$

```python
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import axes3d
from matplotlib import style

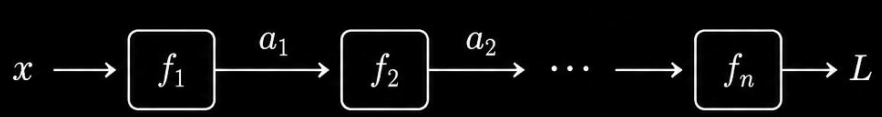
def SGD(samples, y, step_size=2, max_iter_count=1000):
  m, var = samples.shape
  theta = np.zeros(2)
  y = y.flatten()
  #进入循环内
  loss = 1
  iter_count = 0
  iter_list=[]
  loss_list=[]
  theta1=[]
  theta2=[]
  while loss > 0.01 and iter_count < max_iter_count:
    loss = 0
    #梯度计算
    theta1.append(theta[0])
    theta2.append(theta[1])
    #样本维数下标
    rand1 = np.random.randint(0,m,1)
    h = np.dot(theta,samples[rand1].T)
    #关键点，只需要一个样本点来更新权值
    for i in range(len(theta)):
      theta[i] =theta[i] - step_size*(1/m)*(h - y[rand1])*samples[rand1,i]
    #计算总体的损失精度，等于各个样本损失精度之和
    for i in range(m):
      h = np.dot(theta.T, samples[i])
      #每组样本点损失的精度
      every_loss = (1/(var*m))*np.power((h - y[i]), 2)
      loss = loss + every_loss

    print("iter_count: ", iter_count, "the loss:", loss)
```

3.13. Backpropagation

Backpropagation & Chain Rule

1. Computational Graph



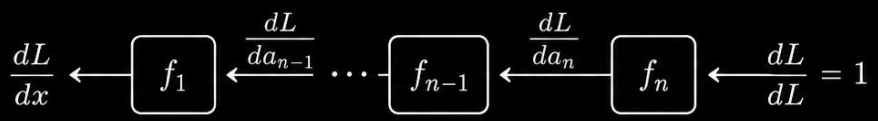
x : input
a_i : intermediate value
L : scalar loss

2. Chain Rule

$$\frac{dL}{dx} = \frac{dL}{da_n}\frac{da_n}{da_{n-1}}\cdots\frac{da_1}{dx}$$

The gradient of the loss w.r.t. x is the product of local derivatives along the computation path.

3. Backpropagation (Reverse Mode)



Gradients flow backward from the loss to the input.

1. Initialize: $\frac{dL}{dL} = 1$
2. For $i = n, n-1, \ldots, 1$]:
$$\frac{dL}{da_{i-1}} = \frac{dL}{da_i} \cdot \frac{da_i}{da_{i-1}}$$
3. Finally, get $\frac{dL}{dx}$

3.14. fused_add_rms_norm_backward_kernel

Load inputs and compute row statistics

```
# z = x + residual
row = tl.program_id(0)
cols = tl.arange(0, BLOCK_SIZE)
mask = cols < n_cols
offsets = row * n_cols + cols
z = tl.load(
    updated_residual_ptr + offsets,
    mask=mask, other=0.0).to(tl.float32)
weight = tl.load(
    weight_ptr + cols,
    mask=mask, other=0.0).to(tl.float32)
grad_out = tl.load(
    grad_out_ptr + offsets,
    mask=mask, other=0.0).to(tl.float32)
grad_z_direct = tl.load(
    grad_updated_residual_ptr + offsets,
    mask=mask, other=0.0).to(tl.float32)
variance = tl.sum(z * z, axis=0) / n_cols
rrms = 1.0 / tl.sqrt(variance + eps)
```

Compute and accumulate gradients

```
scaled_grad = grad_out * weight
dot = tl.sum(scaled_grad * z, axis=0)

# Include the direct residual-output gradient
grad_z = (
    scaled_grad * rrms
    - z * (rrms * rrms * rrms) * dot / n_cols
    + grad_z_direct
)

# z = x + residual, so both gradients match
tl.store(grad_x_ptr + offsets,
         grad_z, mask=mask)
tl.store(grad_residual_ptr + offsets,
         grad_z, mask=mask)

# Accumulate weight gradients across rows
grad_weight = grad_out * z * rrms
tl.atomic_add(grad_weight_ptr + cols,
              grad_weight, mask=mask)
```

3.15. FusedAddRMSNorm

```
class FusedAddRMSNorm(torch.autograd.Function):
    @staticmethod
    def forward(ctx, x, residual, weight, eps=1e-6):
        _check_inputs(x, residual, weight)
        out, updated_residual = fused_add_rms_norm(
            x, residual, weight, eps)
        ctx.save_for_backward(
            updated_residual, weight.contiguous())
        ctx.eps = eps
        return out, updated_residual

    @staticmethod
    def backward(ctx, grad_out,
                 grad_updated_residual):
        updated_residual, weight = ctx.saved_tensors
        n_rows, n_cols = updated_residual.shape
        if grad_out is None:
            grad_out = torch.zeros_like(
                updated_residual)
        if grad_updated_residual is None:
            grad_updated_residual = torch.zeros_like(
                updated_residual)
        # Backward pass, continued
        grad_x = torch.empty_like(updated_residual)
        grad_residual = torch.empty_like(
            updated_residual)
        grad_weight_fp32 = torch.zeros(
            (n_cols,), dtype=torch.float32,
            device=updated_residual.device)

        _fused_add_rms_norm_backward_kernel[
            (n_rows,)
        ](
            updated_residual,
            weight, grad_out.contiguous(),
            grad_updated_residual.contiguous(),
            grad_x, grad_residual, grad_weight_fp32,
            n_cols, ctx.eps,
            BLOCK_SIZE=_block_size(n_cols),
        )
        return (grad_x, grad_residual,
                grad_weight_fp32.to(weight.dtype),
                None)
```

3.16. Advanced Dispatching & Trade-offs

Autograd dispatch: enabling the backward pass

```
# Register the AutogradCUDA implementation
_autograd = torch.library.Library("lab", "IMPL", "AutogradCUDA")
_autograd.impl("fused_add_rms_norm", _autograd_impl)
```

Sequential re-dispatching and trade-offs

```
def _autograd_impl(x, residual, weight, eps=1e-6):
    """Route graph capture and normal execution."""
    if isinstance(x, FakeTensor):
        # Use the Meta path while tracing
        return _meta_impl(x, residual, weight, eps)
    # Use the Triton forward and backward implementations
    return FusedAddRMSNorm.apply(x, residual, weight, eps)
```

Reference

- https://hanlab.mit.edu/courses/2026-fall-65940
- https://novel.ict.ac.cn/aics/
- https://github.com/microsoft/AI-System
- https://github.com/Infrasys-AI/AISystem

African Scientific Research and Innovation Council

BAAI 智源研究院

北京大学 PEKING UNIVERSITY

中国人民大学 RENMIN UNIVERSITY OF CHINA

Thanks

Questions & further discussion welcome

Guojie Luo

gluo@pku.edu.cn

Jing Wang

jwang@ruc.edu.cn

BAAI Team

github.com/flagos-ai

